

数据结构与算法（B） 笔记

Humble

2026 年 6 月 23 日

目录

1	前置知识与算法分析	3
1.1	“计算”的数学模型	3
1.2	图灵机与停机问题	3
1.2.1	图灵机的数学定义	3
1.2.2	停机问题	4
1.2.3	停机问题的不可判定性	4
1.3	数据结构与算法	4
1.4	算法分析简介	5
1.4.1	大 O 记号	5
1.4.2	大 Ω 记号	5
1.4.3	大 Θ 记号	5
1.4.4	举例说明	5
1.5	Python 数据类型的性能	6
1.5.1	<code>list</code> 常见操作的复杂度	6
1.5.2	哈希对象的复杂度	6
1.5.3	什么时候会违反上述复杂度	7
2	线性结构与栈的应用	7
2.1	线性结构	7
2.1.1	栈	8
2.1.2	队列	9
2.1.3	双端队列	10
2.1.4	线性表	11
2.2	表达式转换	13
2.2.1	表达式树	13
2.2.2	记号、优先级与基本工具	14
2.2.3	中缀表达式转后缀表达式	16
2.2.4	中缀表达式转前缀表达式	17
2.2.5	从后缀或前缀表达式还原中缀表达式	18
2.2.6	用表达式树统一转换	19
2.2.7	前缀、后缀表达式求值	22

2.2.8	用表达式树求值	23
2.2.9	小结	24
3	树结构基础与转换	25
3.1	树的基本定义与实现	25
3.1.1	基本定义	25
3.1.2	树的实现	25
3.2	树、二叉树与森林的相互转化	26
3.2.1	1. 树与二叉树之间的转化	26
3.2.2	2. 森林与二叉树之间的转化	30
4	优先队列、Huffman 树与查找树	34
4.1	二叉堆	34
4.1.1	完全二叉树的列表实现	34
4.1.2	二叉堆	34
4.2	Huffman 树	36
4.2.1	1. Huffman 树	36
4.2.2	2. Huffman 编码	39
4.3	二叉查找树与 AVL 树	42
4.3.1	AVL 树	44
5	排序、查找与字符串匹配	50
5.1	排序算法	50
5.1.1	冒泡排序	50
5.1.2	选择排序	51
5.1.3	插入排序	51
5.1.4	希尔排序	52
5.1.5	归并排序	53
5.1.6	快速排序	55
5.2	二分查找	57
5.2.1	Problem 1	57
5.2.2	Problem 2	58
5.2.3	Problem 3	58
5.2.4	更多应用场景	59
5.3	KMP 字符串匹配算法	59
5.3.1	1. KMP 的动机	60
5.3.2	2. LPS 数组	61
5.3.3	3. 最终 KMP 算法	63
6	并查集	64
6.1	基本思想	65
6.2	两个核心优化	65
6.2.1	1. 路径压缩	65
6.2.2	2. 按大小或按秩合并	65

6.3	时间复杂度	65
6.4	使用示例	67
6.5	典型应用：统计无向图的连通分量	68
6.6	小结	68
7	图算法	68
7.1	BFS 与 DFS	71
7.2	拓扑排序	73
7.3	连通分量与强连通分量	74
7.4	最短路径	78
7.4.1	Dijkstra 算法	78
7.4.2	Bellman-Ford 算法	78
7.4.3	Floyd 算法	78
7.5	最小生成树	82
7.5.1	Prim 算法	82
7.5.2	Kruskal 算法	82

1 前置知识与算法分析

1.1 “计算”的数学模型

为了解决现实中的问题，我们常常借助“计算”这一方法。形式化地说，计算过程可以理解成一种**基于有穷观点的能行方法**：它由有限条明确规则构成，并且可以被机械地执行。

一个计算过程通常应满足：

- 由有限数量的明确指令构成；
- 每条指令都可以机械执行；
- 在给定状态下，执行结果是确定的；
- 若算法声称求解某问题，则应在有限步后终止并给出结果。

围绕这种思想，数学家和计算机科学家提出了多种等价的计算模型，包括递归函数、lambda 演算、Post 机与图灵机等。其中图灵机最适合用来讨论“什么问题可被算法解决”。

1.2 图灵机与停机问题

1.2.1 图灵机的数学定义

一个（确定性）图灵机可以定义为七元组 $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{acc}, q_{rej})$ ，其中：

- Q 是有限状态集；
- Σ 是输入字母表，且空白符号 $\sqcup \notin \Sigma$ ；
- Γ 是纸带字母表，满足 $\Sigma \subseteq \Gamma$ 且 $\sqcup \in \Gamma$ ；
- $\delta: (Q \setminus \{q_{acc}, q_{rej}\}) \times \Gamma \rightarrow Q \times \Gamma \times \{L, R, S\}$ 是转移函数；

- $q_0 \in Q$ 是初始状态;
- $q_{acc} \in Q$ 是接受状态;
- $q_{rej} \in Q$ 是拒绝状态, 且 $q_{rej} \neq q_{acc}$ 。

图灵机的配置由当前状态、纸带内容与读写头位置共同决定。若记某个配置为 C , 则由转移函数 δ 可定义一步转移关系 $C \vdash C'$ 。若从初始配置出发, 经过有限次转移后到达接受状态, 则称该输入被机器接受; 若到达拒绝状态, 则称该输入被拒绝。

1.2.2 停机问题

设 $\langle M, w \rangle$ 表示图灵机 M 与输入串 w 的编码。停机问题定义为语言

$$\text{HALT} = \{ \langle M, w \rangle \mid M \text{ 在输入 } w \text{ 上会停机} \}.$$

问题是: 是否存在一个图灵机 H , 对任意输入 $\langle M, w \rangle$ 都能正确判断 M 在 w 上是否停机?

1.2.3 停机问题的不可判定性

下面用反证法证明不存在这样的通用判定机。

假设存在一个图灵机 H , 对任意 $\langle M, w \rangle$, 若 M 在 w 上停机则 H 接受; 若 M 在 w 上不停机则 H 拒绝。

构造一台新图灵机 D , 其输入为某台图灵机 M 的编码 $\langle M \rangle$, 并执行如下过程:

1. 将 $\langle M, \langle M \rangle \rangle$ 输入给 H ;
2. 若 H 接受, 则 D 进入无限循环;
3. 若 H 拒绝, 则 D 停机接受。

现在考虑 D 自身的编码 $\langle D \rangle$ 并令输入也为 $\langle D \rangle$ 。分析 $D(\langle D \rangle)$:

- 若 H 判断 $D(\langle D \rangle)$ 会停机, 则按构造 D 反而进入无限循环, 因此它不停机;
- 若 H 判断 $D(\langle D \rangle)$ 不会停机, 则按构造 D 会停机接受。

这与 H 的正确性假设矛盾, 因此这样的判定机不存在。故停机问题不可判定。

这个结论说明: 计算理论不仅关心“哪些问题能算”, 也关心“哪些问题根本不存在通用算法”。

1.3 数据结构与算法

数据结构研究数据的组织方式。不同组织方式会影响存储、访问、插入、删除、查找和更新效率。常见数据结构包括数组、链表、栈、队列、树、图、堆与散列表等。

算法研究解决问题的步骤。一个算法需要说明输入、输出、执行过程、终止条件与正确性。常见思想包括枚举、递归、分治、贪心、动态规划、回溯、搜索、排序和图算法等。

二者通常要结合起来看。合适的数据结构会让算法更自然、更高效; 算法需求也会反过来决定应选择怎样的数据结构。

1.4 算法分析简介

设算法的输入规模为 n 。算法分析关注的是当 n 增大时，算法所需资源的增长趋势，而不是某台机器上的精确运行秒数。最常见的分析对象是时间复杂度和空间复杂度。

1.4.1 大 O 记号

设 $f(n)$ 和 $g(n)$ 为定义在足够大整数上的非负函数。若存在常数 $c > 0$ 和 n_0 ，使得对一切 $n \geq n_0$ 都有

$$0 \leq f(n) \leq c g(n),$$

则称 $f(n) = O(g(n))$ 。

它表示 f 的增长至多与 g 同阶，常用于给出复杂度上界。

1.4.2 大 Ω 记号

若存在常数 $c > 0$ 和 n_0 ，使得对一切 $n \geq n_0$ 都有

$$0 \leq c g(n) \leq f(n),$$

则称 $f(n) = \Omega(g(n))$ 。

它表示 f 的增长至少与 g 同阶，常用于给出复杂度下界。

1.4.3 大 Θ 记号

若同时有 $f(n) = O(g(n))$ 且 $f(n) = \Omega(g(n))$ ，则称

$$f(n) = \Theta(g(n)).$$

等价地，存在常数 $c_1, c_2 > 0$ 和 n_0 ，使得对一切 $n \geq n_0$ 都有

$$c_1 g(n) \leq f(n) \leq c_2 g(n).$$

它表示 f 与 g 同阶增长。

1.4.4 举例说明

若 $f(n) = 3n + 10$ ，则有 $f(n) = O(n)$ 、 $f(n) = \Omega(n)$ ，因此 $f(n) = \Theta(n)$ 。

若 $f(n) = n^2 + 5n + 7$ ，则有 $f(n) = O(n^2)$ 、 $f(n) = \Omega(n^2)$ ，因此 $f(n) = \Theta(n^2)$ 。

若 $f(n) = \log n$ ，则有 $f(n) = O(n)$ ，但不可能有 $f(n) = \Omega(n)$ 。

分析复杂度时常用以下原则：

- 顺序执行的代码段，总复杂度取增长最快的一项；
- 嵌套循环的复杂度通常相乘；
- 递归算法常结合递归树、递推式或主定理分析；
- 常数因子和低阶项通常被忽略。

1.5 Python 数据类型的性能

Python 内置类型已经封装了许多常见数据结构。理解它们的典型复杂度，有助于避免不必要的低效操作。

1.5.1 list 常见操作的复杂度

操作	平均复杂度	说明
<code>a[i]</code>	$O(1)$	按下标访问
<code>a[i] = x</code>	$O(1)$	按下标赋值
<code>append(x)</code>	均摊 $O(1)$	尾部追加
<code>pop()</code>	均摊 $O(1)$	删除尾部元素
<code>insert(i, x)</code>	$O(n)$	需要移动元素
<code>pop(i)</code>	$O(n)$	需要移动元素
<code>remove(x)</code>	$O(n)$	先查找再删除
<code>x in a</code>	$O(n)$	线性查找
<code>index(x)</code>	$O(n)$	线性查找
<code>count(x)</code>	$O(n)$	遍历计数
<code>extend(b)</code>	$O(k)$	k 为追加的元素个数
<code>reverse()</code>	$O(n)$	原地翻转
<code>sort()</code>	$O(n \log n)$	Python 的 Timsort
切片 <code>a[i:j]</code>	$O(k)$	k 为切片长度
迭代遍历	$O(n)$	按元素顺序访问

1.5.2 哈希对象的复杂度

对于 dict，常见操作的平均复杂度为：

操作	平均复杂度	说明
<code>d[k]</code>	$O(1)$	按键取值
<code>d[k] = v</code>	$O(1)$	插入或更新
<code>del d[k]</code>	$O(1)$	删除键
<code>k in d</code>	$O(1)$	键成员判断
<code>get(k)</code>	$O(1)$	安全取值
<code>keys() / values() / items()</code>	$O(1)$ 生成视图	视图本身的创建是常数时间

操作	平均复杂度	说明
遍历字典	$O(n)$	与元素个数成正比

字典依赖哈希表。对键的访问先计算哈希值，再在对应桶中定位，因此平均情况下查找、插入和删除都很快。`set` 的情形与 `dict` 类似，只是它存储的是元素本身而不是键值对。

1.5.3 什么时候会违反上述复杂度

上面的复杂度通常是平均意义或均摊意义上的结论，以下情况可能使实际表现偏离：

- **哈希冲突严重**：`dict` 或 `set` 中大量键落入同一桶，查找可能退化；
- **恶意输入**：如果键的构造刻意制造冲突，哈希表的性能会明显下降；
- **动态扩容**：`list.append()`、`dict` 插入等操作在触发扩容时会有一次较大的搬移或重哈希开销；
- **元素类型本身开销大**：即使主操作是 $O(1)$ ，若键的哈希计算或比较本身很慢，整体时间也会变大；
- **最坏情况分析**：复杂度表中的平均值不等于最坏值，某些操作在理论上可能退化到更高阶。

因此，复杂度表描述的是典型行为，实际性能还要结合数据规模、数据分布和具体实现一起看。

2 线性结构与栈的应用

2.1 线性结构

线性结构是一种有序数据项的集合，直观而言，数据项之间像一条线一样被前后串接起来：- 除首尾数据项之外，每个数据项都有唯一的前驱与后继；- 新数据项通常加到首端或尾端。

对于线性结构而言，不同数据项之间最重要的是先后的次序关系，而不是绝对位置关系。

下面依次介绍栈、队列、双端队列与线性表四种 ADT，并给出基于链表的标准 Python 实现。

统一结点定义 下面的实现都复用同一个 `Node` 类。`value` 存放数据项，`next` 指向后继结点，`prev` 指向前驱结点。栈、队列和线性表只依赖 `next` 也能完成操作；双端队列需要同时维护 `prev` 和 `next`。

```
class Node():
    def __init__(self, value):
        self.value = value
```

```
self.prev = None
self.next = None
```

2.1.1 栈

ADT 定义 栈 (stack) 只允许在同一端插入和删除元素, 遵循后进先出 (LIFO)。基本操作通常包括: `push`、`pop`、`peek`、`isEmpty`、`size`。

链式结构 用单链表实现时, 把表头当作栈顶。`push` 和 `pop` 都只操作表头, 因此时间复杂度都是 $O(1)$ 。

```
class LinkedStack():
    def __init__(self):
        self.top = None
        self._size = 0

    def push(self, value):
        node = Node(value)
        node.next = self.top
        if self.top is not None:
            self.top.prev = node
        self.top = node
        self._size += 1

    def pop(self):
        if self._size == 0:
            raise IndexError("Pop from empty stack!")
        node = self.top
        self.top = node.next
        if self.top is not None:
            self.top.prev = None
        self._size -= 1
        return node.value

    def peek(self):
        if self.top is None:
            raise IndexError("Stack is empty!")
        return self.top.value

    def size(self):
        return self._size

    def isEmpty(self):
        return self._size == 0
```


2.1.2 队列

ADT 定义 队列 (queue) 只允许在一端插入、另一端删除元素, 遵循先进先出 (FIFO)。基本操作通常包括: enqueue、dequeue、front、isEmpty、size。

链式结构 用单链表并维护头尾指针: 入队发生在尾部, 出队发生在头部, 因此两者都是 $O(1)$ 。

```
class LinkedQueue():
    def __init__(self):
        self._size = 0
        self._front = None
        self.rear = None

    def enqueue(self, value):
        node = Node(value)
        if self._size == 0:
            self._front = node
            self.rear = node
        else:
            self.rear.next = node
            node.prev = self.rear
            self.rear = node
        self._size += 1

    def dequeue(self):
        if self._size == 0:
            raise IndexError("Dequeue from an empty queue!")
        node = self._front
        self._front = node.next
        if self._front is None:
            self.rear = None
        else:
            self._front.prev = None
        self._size -= 1
        return node.value

    def front(self):
        if self._front is None:
            raise IndexError("Queue is empty!")
        return self._front.value

    def size(self):
        return self._size

    def isEmpty(self):
        return self._size == 0
```

2.1.3 双端队列

ADT 定义 双端队列 (deque) 允许在两端插入和删除元素。基本操作通常包括: addFront、addRear、removeFront、removeRear、peekFront、peekRear、isEmpty、size。

链式结构 用双向链表最合适, 因为两端操作都只需要改动少量指针。

```
class LinkedDeque:
    def __init__(self, iterable=None):
        self._size = 0
        self.front = None
        self.rear = None
        if iterable is not None:
            for item in iterable:
                self.addRear(item)

    def addFront(self, item):
        node = Node(item)
        if self._size == 0:
            self.front = node
            self.rear = node
        else:
            node.next = self.front
            self.front.prev = node
            self.front = node
        self._size += 1

    def addRear(self, item):
        node = Node(item)
        if self._size == 0:
            self.front = node
            self.rear = node
        else:
            self.rear.next = node
            node.prev = self.rear
            self.rear = node
        self._size += 1

    def removeFront(self):
        if self._size == 0:
            raise IndexError("Remove from an empty deque!")
        node = self.front
        self.front = node.next
        if self.front is None:
            self.rear = None
        else:
            self.front.prev = None
        self._size -= 1
```

```

        return node.value

def removeRear(self):
    if self._size == 0:
        raise IndexError("Remove from an empty deque!")
    node = self.rear
    self.rear = node.prev
    if self.rear is None:
        self.front = None
    else:
        self.rear.next = None
    self._size -= 1
    return node.value

def peekFront(self):
    if self.front is None:
        raise IndexError("Deque is empty!")
    return self.front.value

def peekRear(self):
    if self.rear is None:
        raise IndexError("Deque is empty!")
    return self.rear.value

def __iter__(self):
    current = self.front
    while current is not None:
        yield current.value
        current = current.next

def size(self):
    return self._size

def isEmpty(self):
    return self._size == 0

```

2.1.4 线性表

ADT 定义 线性表 (list) 是由 $n \geq 0$ 个元素组成的有序序列。这里实现的是无序线性表，即元素没有按大小排序，但仍按插入后的链式次序存放。基本操作通常包括：`add(item)`、`search(item)`、`remove(item)`、`isEmpty()`、`size()`。

链式结构 无序线性表可以用单链表实现。每个结点保存一个元素和指向下一结点的引用，线性表本身只需要保存头结点 `head` 与元素个数。

若采用头插法，`add` 只需修改新结点和头指针，时间复杂度为 $O(1)$ ；`search` 和 `remove` 需要从头开始顺序查找，最坏时间复杂度为 $O(n)$ 。删除结点时还要记录当前结点的前驱，才能把前驱的 `next` 越过待删结点。

```
class UnorderedList:
    def __init__(self):
        self.head = None
        self._size = 0

    def add(self, item):
        node = Node(item)
        node.next = self.head
        if self.head is not None:
            self.head.prev = node
        self.head = node
        self._size += 1

    def remove(self, item):
        current = self.head
        previous = None
        found = False
        while current is not None and not found:
            if current.value == item:
                found = True
            else:
                previous = current
                current = current.next
        if not found:
            raise ValueError("Item not found in the list!")
        if previous is None:
            self.head = current.next
            if self.head is not None:
                self.head.prev = None
        else:
            previous.next = current.next
            if current.next is not None:
                current.next.prev = previous
        self._size -= 1

    def search(self, item):
        current = self.head
        found = False
        while current is not None and not found:
            if current.value == item:
                found = True
            else:
                current = current.next
        return found

    def size(self):
        return self._size

    def isEmpty(self):
```

```
return self._size == 0
```

2.2 表达式转换

栈的一个重要应用是对表达式进行转换。例如，将中缀表达式转换为后缀表达式，使之便于计算机处理。在前缀表达式和后缀表达式中，运算顺序已经由符号的位置隐含确定，不再依赖复杂的括号。

常见的三种表达式写法如下：

类型	运算符位置	例子
前缀表达式	运算符在操作数前面	* + A B C
中缀表达式	运算符在两个操作数中间	(A + B) * C
后缀表达式	运算符在操作数后面	A B + C *

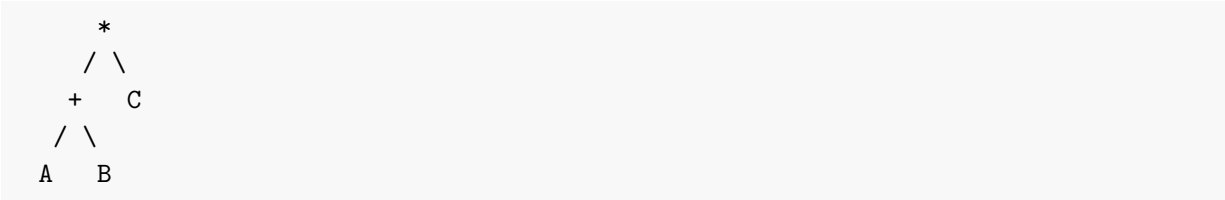
中缀表达式最符合人类习惯，但需要处理优先级和括号；前缀、后缀表达式更适合计算机用栈直接求值。

2.2.1 表达式树

任意一个表达式都可以视作一棵表达式树的某种遍历结果。例如，

```
(A + B) * C
```

对应的表达式树是



三种遍历方式分别对应三种表达式：

前序遍历：根 → 左 → 右

```
* + A B C
```

中序遍历：左 → 根 → 右

```
(A + B) * C
```

后序遍历：左 → 右 → 根

```
A B + C *
```

因此，表达式转换可以被理解为两步：先把表达式还原成树，再对树做不同遍历。

2.2.2 记号、优先级与基本工具

下面的代码约定：

- 支持二元运算符 $+$ $-$ $*$ $/$ $^$ ；
- $^$ 表示乘方，并且是右结合的，即 $2 \wedge 3 \wedge 2$ 等价于 $2 \wedge (3 \wedge 2)$ ；
- 中缀表达式可以写成普通字符串，如 $(A + B) * C$ ；
- 前缀、后缀表达式默认用空格分隔 token，如 $* + A B C$ ；
- 为了让主线更清晰，这里主要处理二元运算。负数数字字面量如 -3 可以识别，但 $-x$ 这类一元负号不单独处理。

```
OPERATORS = {
    "+": (1, "L"),
    "-": (1, "L"),
    "*": (2, "L"),
    "/": (2, "L"),
    "^": (3, "R"),
}

def is_operator(token):
    return token in OPERATORS

def is_number(token):
    try:
        float(token)
        return True
    except ValueError:
        return False

def is_operand(token):
    return token not in OPERATORS and token not in {"(", ")" }

def tokenize(expr):
    """ 把中缀表达式字符串切分成 token 列表。 """
    tokens = []
    i = 0
    prev = None

    while i < len(expr):
        ch = expr[i]

        if ch.isspace():
            i += 1
            continue

        is_negative_number = (
```

```

        ch == "-"
        and (prev is None or prev in OPERATORS or prev == "(")
        and i + 1 < len(expr)
        and (expr[i + 1].isdigit() or expr[i + 1] == ".")
    )

    if ch.isdigit() or ch == "." or is_negative_number:
        start = i
        if is_negative_number:
            i += 1

        dot_count = 0
        while i < len(expr) and (expr[i].isdigit() or expr[i] == "."):
            if expr[i] == ".":
                dot_count += 1
            if dot_count > 1:
                raise ValueError(f" 非法数字: {expr[start:i + 1]}")
            i += 1

        tokens.append(expr[start:i])
        prev = "operand"
        continue

    if ch.isalpha() or ch == "_":
        start = i
        while i < len(expr) and (expr[i].isalnum() or expr[i] == "_"):
            i += 1
        tokens.append(expr[start:i])
        prev = "operand"
        continue

    if ch in "+-*/^()":
        tokens.append(ch)
        prev = ch
        i += 1
        continue

    raise ValueError(f" 无法识别的字符: {ch}")

    return tokens

tokenize("(A + B) * C")

```

输出:

```
['(', 'A', '+', 'B', ')', '*', 'C']
```

2.2.3 中缀表达式转后缀表达式

中缀表达式转后缀表达式常用 **调度场算法** (shunting-yard algorithm)。核心思想是：

- 操作数直接输出；
- 左括号入栈；
- 右括号触发弹栈，直到遇到左括号；
- 运算符入栈前，先把栈顶优先级更高的运算符弹出；如果二者优先级相同，还要看当前运算符是否左结合，相同优先级左结合的运算符从左到右计算，因此应当弹出栈顶运算符；
- 扫描结束后，把栈中剩余运算符依次弹出。

这相当于把中缀表达式中隐含的运算顺序显式地移动到运算符的位置上。

```
def infix_to_postfix(expr):
    tokens = tokenize(expr) if isinstance(expr, str) else list(expr)
    output = []
    stack = []

    for token in tokens:
        if is_operand(token):
            output.append(token)
        elif token == "(":
            stack.append(token)
        elif token == ")":
            while stack and stack[-1] != "(":
                output.append(stack.pop())
            if not stack:
                raise ValueError(" 括号不匹配")
            stack.pop()
        elif is_operator(token):
            while (
                stack
                and stack[-1] in OPERATORS
                and (
                    (OPERATORS[token][0] < OPERATORS[stack[-1]][0])
                    or (
                        OPERATORS[token][0] == OPERATORS[stack[-1]][0]
                        and OPERATORS[token][1] == "L"
                    )
                )
            ):
                output.append(stack.pop())
            stack.append(token)
        else:
            raise ValueError(f" 无法识别的 token: {token}")

    while stack:
        if stack[-1] in "()":
```



```

        raise ValueError(" 括号不匹配")
    output.append(stack.pop())

    return output

postfix = infix_to_postfix("(A + B) * C")
postfix, " ".join(postfix)

```

输出:

```
(['A', 'B', '+', 'C', '*'], 'A B + C *')
```

2.2.4 中缀表达式转前缀表达式

中缀表达式转前缀表达式与中缀表达式转后缀表达式的思路很相近，但是需要注意的是，由于操作符在操作数之前，我们应当从右往左扫描 token 序列：- 操作数直接添加到输出列表中；- 右括号入栈；- 左括号触发弹栈，直至遇到下一个右括号；- 运算符入栈前，如果其优先级低于栈顶运算符的优先级，则应当弹栈；- 如果其优先级与栈顶运算符优先级相同，且该运算符是右结合的，则应当先弹栈；- 扫描结束后，依次弹出栈中剩余操作符。

```

def infix_to_prefix(expr):
    tokens = tokenize(expr) if isinstance(expr, str) else list(expr)
    output = []
    stack = []

    for token in tokens[::-1]:
        if is_operand(token):
            output.append(token)
        elif token == ")":
            stack.append(token)
        elif token == "(":
            while stack and stack[-1] != ")":
                output.append(stack.pop())
            if not stack:
                raise ValueError(" 括号不匹配")
            stack.pop()
        elif is_operator(token):
            while (
                stack
                and stack[-1] in OPERATORS
                and (
                    OPERATORS[token][0] < OPERATORS[stack[-1]][0]
                    or (
                        OPERATORS[token][0] == OPERATORS[stack[-1]][0]
                        and OPERATORS[token][1] == "R"
                    )
                )
            ):
                output.append(stack.pop())
            stack.append(token)

    while stack:
        output.append(stack.pop())

    return output

```

```

        )
    ):
        output.append(stack.pop())
        stack.append(token)
    else:
        raise ValueError(f" 无法识别的 token: {token}")

while stack:
    if stack[-1] in "()":
        raise ValueError(" 括号不匹配")
    output.append(stack.pop())

return output[::-1]

prefix = infix_to_prefix("(A + B) * C")
prefix, " ".join(prefix)

```

输出:

```
(['*', '+', 'A', 'B', 'C'], '* + A B C')
```

2.2.5 从后缀或前缀表达式还原中缀表达式

前缀、后缀表达式不需要括号，是因为每个运算符都能找到自己的两个操作数。利用栈可以直接把它们还原为带括号的中缀表达式。

后缀表达式从左到右扫描：

- 遇到操作数，压栈；
- 遇到运算符，弹出两个表达式，先弹出的是右操作数，后弹出的是左操作数；
- 合成新的中缀表达式后再压栈。

前缀表达式可以反过来从右到左扫描：

- 遇到操作数，压栈；
- 遇到运算符，弹出两个表达式，先弹出的是左操作数，后弹出的是右操作数；
- 合成新的中缀表达式后再压栈。

```

def postfix_to_infix(expr):
    tokens = expr.split() if isinstance(expr, str) else list(expr)
    stack = []

    for token in tokens:
        if is_operand(token):
            stack.append(token)
        elif is_operator(token):
            if len(stack) < 2:
                raise ValueError(" 表达式不完整")
            right = stack.pop()
            left = stack.pop()

```

```

        stack.append(f"({left} {token} {right})")

    if len(stack) != 1:
        raise ValueError(" 表达式不合法")
    return stack[0]

def prefix_to_infix(expr):
    tokens = expr.split() if isinstance(expr, str) else list(expr)
    stack = []

    for token in reversed(tokens):
        if is_operand(token):
            stack.append(token)
        elif is_operator(token):
            if len(stack) < 2:
                raise ValueError(" 表达式不完整")
            left = stack.pop()
            right = stack.pop()
            stack.append(f"({left} {token} {right})")

    if len(stack) != 1:
        raise ValueError(" 表达式不合法")
    return stack[0]

postfix_to_infix("A B + C *"), prefix_to_infix("* + A B C")

```

输出：

```
('((A + B) * C)', '((A + B) * C)')
```

2.2.6 用表达式树统一转换

如果直接写“前缀转后缀”“后缀转前缀”“中缀转前缀”等函数，很容易出现大量相似代码。表达式树可以把转换过程简化为：

任意一种表达式 -> 表达式树 -> 任意一种遍历结果

于是只需要实现三类“建树”和三类“遍历”，就能覆盖所有转换。

```
from dataclasses import dataclass
```

```

@dataclass
class ExprNode:
    value: str
    left: object = None
    right: object = None

```

```
def is_leaf(self):
    return self.left is None and self.right is None

def tree_from_postfix(expr):
    tokens = expr.split() if isinstance(expr, str) else list(expr)
    stack = []

    for token in tokens:
        if is_operand(token):
            stack.append(ExprNode(token))
        elif is_operator(token):
            if len(stack) < 2:
                raise ValueError(" 表达式不完整")
            right = stack.pop()
            left = stack.pop()
            stack.append(ExprNode(token, left, right))

    if len(stack) != 1:
        raise ValueError(" 表达式不合法")
    return stack[0]

def tree_from_prefix(expr):
    tokens = expr.split() if isinstance(expr, str) else list(expr)
    stack = []

    for token in reversed(tokens):
        if is_operand(token):
            stack.append(ExprNode(token))
        elif is_operator(token):
            if len(stack) < 2:
                raise ValueError(" 表达式不完整")
            left = stack.pop()
            right = stack.pop()
            stack.append(ExprNode(token, left, right))

    if len(stack) != 1:
        raise ValueError(" 表达式不合法")
    return stack[0]

def tree_from_infix(expr):
    return tree_from_postfix(infix_to_postfix(expr))

def tree_to_prefix(node):
    if node.is_leaf():
        return [node.value]
    return [node.value] + tree_to_prefix(node.left) + tree_to_prefix(node.right)
```

```

def tree_to_postfix(node):
    if node.is_leaf():
        return [node.value]
    return tree_to_postfix(node.left) + tree_to_postfix(node.right) + [node.value]

def tree_to_infix(node):
    if node.is_leaf():
        return node.value
    return f"({tree_to_infix(node.left)} {node.value} {tree_to_infix(node.right)})"

def build_tree(expr, notation):
    if notation == "infix":
        return tree_from_infix(expr)
    if notation == "prefix":
        return tree_from_prefix(expr)
    if notation == "postfix":
        return tree_from_postfix(expr)
    raise ValueError("notation 必须是 infix、prefix 或 postfix")

def convert(expr, source, target):
    tree = build_tree(expr, source)
    if target == "infix":
        return tree_to_infix(tree)
    if target == "prefix":
        return " ".join(tree_to_prefix(tree))
    if target == "postfix":
        return " ".join(tree_to_postfix(tree))
    raise ValueError("target 必须是 infix、prefix 或 postfix")

```

```
expr = "(A + B) * C"
```

```

print("infix -> prefix :", convert(expr, "infix", "prefix"))
print("infix -> postfix:", convert(expr, "infix", "postfix"))
print("prefix -> infix  :", convert("* + A B C", "prefix", "infix"))
print("postfix-> prefix :", convert("A B + C *", "postfix", "prefix"))

```

输出:

```

infix -> prefix : * + A B C
infix -> postfix: A B + C *
prefix -> infix  : ((A + B) * C)
postfix-> prefix : * + A B C

```

2.2.7 前缀、后缀表达式求值

后缀表达式求值和后缀还原中缀非常相似，只是栈中保存的不再是字符串，而是数值：

- 遇到数字或变量，取出它的值并压栈；
- 遇到运算符，弹出两个数，计算结果后再压栈；
- 扫描结束后，栈中唯一元素就是表达式的值。

前缀表达式同理，只是从右往左扫描。中缀表达式可以先转成后缀再求值，也可以先建表达式树再递归求值。

```
def apply_operator(op, left, right):
    if op == "+":
        return left + right
    if op == "-":
        return left - right
    if op == "*":
        return left * right
    if op == "/":
        return left / right
    if op == "^":
        return left ** right
    raise ValueError(f" 未知运算符: {op}")

def value_of(token, env=None):
    if is_number(token):
        value = float(token)
        return int(value) if value.is_integer() else value

    env = env or {}
    if token in env:
        return env[token]

    raise ValueError(f" 变量 {token} 没有给定取值")

def eval_postfix(expr, env=None):
    tokens = expr.split() if isinstance(expr, str) else list(expr)
    stack = []

    for token in tokens:
        if is_operand(token):
            stack.append(value_of(token, env))
        elif is_operator(token):
            if len(stack) < 2:
                raise ValueError(" 表达式不完整")
            right = stack.pop()
            left = stack.pop()
            stack.append(apply_operator(token, left, right))
```

```

    if len(stack) != 1:
        raise ValueError(" 表达式不合法")
    return stack[0]

def eval_prefix(expr, env=None):
    tokens = expr.split() if isinstance(expr, str) else list(expr)
    stack = []

    for token in reversed(tokens):
        if is_operand(token):
            stack.append(value_of(token, env))
        elif is_operator(token):
            if len(stack) < 2:
                raise ValueError(" 表达式不完整")
            left = stack.pop()
            right = stack.pop()
            stack.append(apply_operator(token, left, right))

    if len(stack) != 1:
        raise ValueError(" 表达式不合法")
    return stack[0]

eval_postfix("3 4 + 5 *"), eval_prefix("* + 3 4 5")

```

输出：

```
(35, 35)
```

2.2.8 用表达式树求值

表达式树求值更接近递归定义：

- 如果当前结点是叶子结点，它就是操作数，直接取值；
- 如果当前结点是运算符，就先求左子树的值，再求右子树的值，最后把二者合并。

这说明表达式树不仅能统一转换，也能统一求值。

```

def eval_tree(node, env=None):
    if node.is_leaf():
        return value_of(node.value, env)

    left = eval_tree(node.left, env)
    right = eval_tree(node.right, env)
    return apply_operator(node.value, left, right)

def evaluate(expr, notation="infix", env=None):
    tree = build_tree(expr, notation)

```

```

return eval_tree(tree, env)

print(evaluate("(3 + 4) * 5", "infix"))
print(evaluate("* + A B C", "prefix", {"A": 3, "B": 4, "C": 5}))
print(evaluate("A B + C *", "postfix", {"A": 3, "B": 4, "C": 5}))

```

输出：

```

35
35
35

```

2.2.9 小结

- 中缀表达式适合人读，但程序处理时要考虑括号、优先级和结合性；
- 后缀表达式适合从左到右用栈求值；
- 前缀表达式适合从右到左用栈求值；
- 表达式树把三种写法统一起来：前缀是前序遍历，中缀是中序遍历，后缀是后序遍历；
- 用树做中间表示后，任意表达式转换都可以写成“建树 + 遍历”，求值也可以写成对树的递归。

```

# 综合例子
expr = "3 + 4 * 2 / (1 - 5) ^ 2 ^ 3"

prefix = convert(expr, "infix", "prefix")
postfix = convert(expr, "infix", "postfix")

print(" 中缀:", expr)
print(" 前缀:", prefix)
print(" 后缀:", postfix)
print(" 值  :", evaluate(expr, "infix"))

```

输出：

```

中缀: 3 + 4 * 2 / (1 - 5) ^ 2 ^ 3
前缀: + 3 / * 4 2 ^ - 1 5 ^ 2 3
后缀: 3 4 2 * 1 5 - 2 3 ^ ^ / +
值   : 3.0001220703125

```


3 树结构基础与转换

3.1 树的基本定义与实现

3.1.1 基本定义

- 节点 Node 每个节点有“键值”。
- 边 Edge 每条边连接两个节点，表示节点之间的关联；除根节点之外的每个节点恰有一条来自另一节点的入边；每个节点可以有多条指向其他节点的出边。
- 根 Root 树中唯一一个没有入边的节点。
- 路径 Path
- 子节点 Children
- 父节点 Parent
- 兄弟节点 Sibling
- 子树 Subtree
- 叶节点 Leaf 没有子节点的节点。
- 层级 Level 从根节点开始到达某个节点的路径所包含边的数量。根节点的层级为 0。
- 高度 Height 树的节点的最大层级数。

3.1.2 树的实现

树通常有两种简单的实现方法：一种是利用嵌套列表，另一种是利用链表。在这里我们主要介绍链表实现的二叉树。

```
class BinaryTree:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None

    def insertLeft(self, value):
        newNode = BinaryTree(value)
        if self.left is None:
            self.left = newNode
        else:
            newNode.left = self.left
            self.left = newNode

    def insertRight(self, value):
        newNode = BinaryTree(value)
        if self.right is None:
            self.right = newNode
        else:
            newNode.right = self.right
            self.right = newNode
```

3.2 树、二叉树与森林的相互转化

普通树中的一个节点可以有任意多个孩子，而二叉树中的一个节点最多只有左、右两个孩子。为了用二叉树表示普通树或森林，需要重新解释二叉树中两个指针的含义。最常用的方法是**左孩子右兄弟表示法**，也叫 **Left-Child Right-Sibling, LCRS** 表示法。

它的核心原则只有一句话：

- **left** 指向该节点在原树中的**第一个孩子**；
- **right** 指向该节点在原树中的**下一个兄弟**。

因此，转换后的二叉树并不是普通意义上“左子树小、右子树大”或“左右孩子都是真正子节点”的二叉树。这里的 **right** 指针更多表示同一层中的兄弟关系，而不是原树中的父子关系。

3.2.1 1. 树与二叉树之间的转化

设普通树中某个节点 **A** 有若干个孩子，按从左到右的顺序分别为：

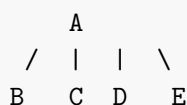
A 的孩子：B, C, D, E

按照“左孩子右兄弟”原则转化为二叉树时：

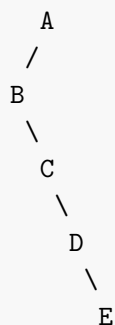
1. **A.left = B**，也就是把第一个孩子 **B** 放到 **A** 的左指针上。
2. **B.right = C**，因为 **C** 是 **B** 的下一个兄弟。
3. **C.right = D**，因为 **D** 是 **C** 的下一个兄弟。
4. **D.right = E**，因为 **E** 是 **D** 的下一个兄弟。
5. 对 **B, C, D, E** 各自的子树继续递归执行同样的规则。

转换后的局部结构可以理解为：

普通树：



二叉树表示：



需要注意：在这个二叉树中，**B, C, D, E** 仍然都是普通树里 **A** 的孩子，只是除了

第一个孩子 B 通过 `A.left` 连接外，其余孩子都挂在右兄弟链上。

普通树转二叉树的步骤 对普通树中每个节点执行以下操作：

1. 如果节点没有孩子，则它在二叉树中的 `left` 指针为空。
2. 如果节点有孩子，则第一个孩子成为它的 `left` 节点。
3. 从第二个孩子开始，依次作为前一个孩子的 `right` 节点，形成一条右兄弟链。
4. 对每个孩子节点递归处理它自己的孩子列表。

二叉树还原为普通树的步骤 如果一棵二叉树本来就是由普通树按 LCRS 方法转换得到的，那么可以反向还原：

1. 对于当前二叉树节点 A，它对应普通树中的同名节点 A。
2. 沿着 `A.left` 找到它的第一个孩子。
3. 从这个第一个孩子开始，不断沿 `right` 指针向右走，得到 A 的全部孩子。
4. 对这些孩子递归还原它们自己的孩子。
5. 如果正在还原的是一棵普通树，那么根节点的 `right` 指针应该为空；如果根节点的 `right` 不为空，说明它更像是在表示一个森林。

```
from __future__ import annotations

from dataclasses import dataclass, field
from typing import Any

@dataclass
class TreeNode:
    """ 普通树节点：一个节点可以有任意多个孩子。 """

    value: Any
    children: list["TreeNode"] = field(default_factory=list)

    def add_child(self, child: "TreeNode") -> "TreeNode":
        self.children.append(child)
        return child

@dataclass
class BinaryNode:
    """ 二叉树节点：left 表示第一个孩子，right 表示下一个兄弟。 """

    value: Any
    left: "BinaryNode | None" = None
    right: "BinaryNode | None" = None

def _encode_tree_node(root: TreeNode | None) -> BinaryNode | None:
    """ 把一棵普通树编码为 LCRS 二叉树，不处理根节点的兄弟。 """
```

```

if root is None:
    return None

binary_root = BinaryNode(root.value)

if root.children:
    binary_root.left = _encode_tree_node(root.children[0])
    current = binary_root.left

    for child in root.children[1:]:
        current.right = _encode_tree_node(child)
        current = current.right

return binary_root

def tree_to_binary(root: TreeNode | None) -> BinaryNode | None:
    """ 普通树 -> 二叉树。 """

    return _encode_tree_node(root)

def _decode_tree_node(root: BinaryNode | None) -> TreeNode | None:
    """ 把一个 LCRS 二叉树节点还原为普通树节点，不处理该节点自己的兄弟。 """

    if root is None:
        return None

    tree_root = TreeNode(root.value)
    child = root.left

    while child is not None:
        tree_root.children.append(_decode_tree_node(child))
        child = child.right

    return tree_root

def binary_to_tree(root: BinaryNode | None) -> TreeNode | None:
    """ 二叉树 -> 普通树。要求根节点没有右兄弟。 """

    if root is not None and root.right is not None:
        raise ValueError(" 根节点存在 right 指针，当前二叉树表示的是森林，请使用  
binary_to_forest")

    return _decode_tree_node(root)

def print_general_tree(root: TreeNode | None, indent: str = "") -> None:

```

```

""" 按缩进打印普通树。 """

if root is None:
    print("<empty>")
    return

print(f"{indent}{root.value}")
for child in root.children:
    print_general_tree(child, indent + " ")

def print_lcrs_binary_tree(root: BinaryNode | None, indent: str = "", label: str =
"root") -> None:
    """ 打印 LCRS 二叉树, 并标明 left/right 的含义。 """

    if root is None:
        print(f"{indent}{label}: <empty>")
        return

    print(f"{indent}{label}: {root.value}")
    if root.left is not None:
        print_lcrs_binary_tree(root.left, indent + " ", "left(first child)")
    if root.right is not None:
        print_lcrs_binary_tree(root.right, indent + " ", "right(next sibling)")

```

```

# 示例: 普通树 -> 二叉树 -> 普通树
#
# 普通树结构:
#      A
#     / | \
#    B  C  D
#   / \   /
#  E  F  G

root = TreeNode("A")
node_b = root.add_child(TreeNode("B"))
root.add_child(TreeNode("C"))
node_d = root.add_child(TreeNode("D"))
node_b.add_child(TreeNode("E"))
node_b.add_child(TreeNode("F"))
node_d.add_child(TreeNode("G"))

binary_root = tree_to_binary(root)
restored_root = binary_to_tree(binary_root)

print(" 原普通树: ")
print_general_tree(root)

print("\n转换后的 LCRS 二叉树: ")

```

```
print_lcrs_binary_tree(binary_root)

print("\n还原后的普通树: ")
print_general_tree(restored_root)
```

输出:

原普通树:

```
A
  B
    E
    F
  C
  D
    G
```

转换后的 LCRS 二叉树:

```
root: A
  left(first child): B
    left(first child): E
      right(next sibling): F
    right(next sibling): C
      right(next sibling): D
        left(first child): G
```

还原后的普通树:

```
A
  B
    E
    F
  C
  D
    G
```

3.2.2 2. 森林与二叉树之间的转化

森林是若干棵互不相交的树的集合。设一个森林由三棵树组成，它们的根节点分别是：

```
T1 的根: A
T2 的根: H
T3 的根: K
```

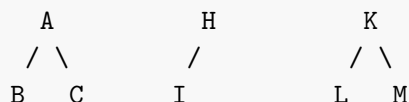
森林转二叉树时，仍然使用“左孩子右兄弟”原则，只是需要把不同树的根节点也看成同一层的兄弟：

1. 先把每棵普通树分别按 LCRS 方法转换成二叉树。
2. 第一棵树的根 A 作为整棵二叉树的根。
3. 第二棵树的根 H 作为 A.right，表示 H 是 A 的下一个兄弟。

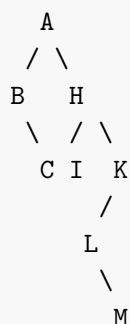
4. 第三棵树的根 K 作为 H.right，继续形成右兄弟链。
5. 每棵树根节点自己的孩子仍然通过 left 指针进入。

也就是说，森林中的多棵树，在二叉树中会表现为一条从根节点开始的 right 链：

森林：



二叉树表示：



在这张二叉树里：

- A.right = H 表示森林中第二棵树的根；
- H.right = K 表示森林中第三棵树的根；
- A.left, H.left, K.left 分别进入每棵树自己的第一个孩子；
- 根节点之间的 right 链表示森林中不同树根之间的兄弟关系。

森林转二叉树的步骤

1. 如果森林为空，则转换结果为空二叉树。
2. 把森林中第一棵树的根作为二叉树的根。
3. 对每棵树内部按“第一个孩子放 left，其余孩子放 right 兄弟链”的方式转换。
4. 把森林中第二棵、第三棵、后续每棵树的根依次连接到前一棵树根的 right 指针上。

二叉树还原为森林的步骤

1. 从二叉树根节点开始，沿着 right 指针依次访问，得到森林中每棵树的根。
2. 对每个根节点，沿着它的 left 指针进入第一个孩子。
3. 对每个孩子，再沿着 right 指针找到它的所有兄弟，即得到同一个父节点的全部孩子。
4. 递归还原所有子树。

普通树与森林的区别主要体现在根节点的 right 指针上：普通树转换成的二叉树根节点通常没有 right，而森林转换成的二叉树根节点可能有 right，用来连接森林中的下一棵树。

```
def forest_to_binary(forest: list[TreeNode]) -> BinaryNode | None:
    """ 森林 -> 二叉树。森林中各棵树的根节点通过 right 指针连接。 """

    if not forest:
        return None

    binary_root = _encode_tree_node(forest[0])
    current = binary_root

    for tree_root in forest[1:]:
        current.right = _encode_tree_node(tree_root)
        current = current.right

    return binary_root

def binary_to_forest(root: BinaryNode | None) -> list[TreeNode]:
    """ 二叉树 -> 森林。二叉树根节点及其 right 链对应森林的各棵树根。 """

    forest = []
    current = root

    while current is not None:
        forest.append(_decode_tree_node(current))
        current = current.right

    return forest

def print_forest(forest: list[TreeNode]) -> None:
    """ 按顺序打印森林中的每棵树。 """

    if not forest:
        print("<empty forest>")
        return

    for index, tree_root in enumerate(forest, start=1):
        print(f"Tree {index}:")
        print_general_tree(tree_root, " ")
```

示例：森林 -> 二叉树 -> 森林

#

森林结构：

#

```
#      A      H      K
#     / \    /    / \
#    B  C  I   L  M
```

```
tree1 = TreeNode("A", [TreeNode("B"), TreeNode("C")])
```



```

tree2 = TreeNode("H", [TreeNode("I")])
tree3 = TreeNode("K", [TreeNode("L"), TreeNode("M")])

forest = [tree1, tree2, tree3]
binary_forest_root = forest_to_binary(forest)
restored_forest = binary_to_forest(binary_forest_root)

print(" 原森林: ")
print_forest(forest)

print("\n转换后的 LCRS 二叉树: ")
print_lcrs_binary_tree(binary_forest_root)

print("\n还原后的森林: ")
print_forest(restored_forest)

```

输出:

```

原森林:
Tree 1:
  A
    B
    C
Tree 2:
  H
    I
Tree 3:
  K
    L
    M

转换后的 LCRS 二叉树:
root: A
  left(first child): B
    right(next sibling): C
  right(next sibling): H
    left(first child): I
      right(next sibling): K
        left(first child): L
          right(next sibling): M

还原后的森林:
Tree 1:
  A
    B
    C
Tree 2:
  H
    I
Tree 3:

```

K
L
M

4 优先队列、Huffman 树与查找树

4.1 二叉堆

优先队列是一种特殊的队列数据结构，与普通队列的“先进先出 (FIFO)”不同，优先队列中的元素会依据优先级（元素的大小，或是其他全序关系）：对于最大优先队列，优先级最高的元素先出队；最小优先队列则相反。在以下介绍中，我们主要考虑最小优先队列，相关的算法及实现对于最大优先队列几乎是一样的。

与队列一致，优先队列需要支持 `push`、`pop`、`top`、`is_empty`、`size` 等操作。

为了实现一个高效的优先队列数据结构，我们首先介绍完全二叉树的列表实现。

4.1.1 完全二叉树的列表实现

不同于一般的树结构，完全二叉树由于其结构的特殊性，我们简单地用列表进行储存与操作。

对于一棵给定的完全二叉树，我们对其进行层次遍历 (BFS)，得到一个连续的序列，我们将这个序列存储在列表 `lst` 中就得到了完全二叉树的列表实现。

我们首先对这个列表进行较为精细的研究。对于完全二叉树第 k 层的第 i 个元素 (注： k 与 i 均从 0 开始计算)，它在列表中的序号为

$$2^0 + 2^1 + \dots + 2^{k-1} + i = 2^k + i - 1$$

这个节点的左/右子节点 (如有) 分别为完全二叉树第 $k+1$ 层的第 $2i$ 与第 $2i+1$ 个元素。它们在列表中的序号分别为 $2^{k+1} + 2i - 1$ 与 $2^{k+1} + 2i$ 。

因此，经过简单的计算，我们可以得到结论：对于完全二叉树的元素 `lst[k]`，它的父节点为 `lst[(k-1)//2]`，它的左右子节点分别为 `lst[2*k+1]` 与 `lst[2*k+2]`。

4.1.2 二叉堆

堆是一种特殊的完全二叉树，分为最大堆与最小堆。最大堆的每个节点的值不小于它的子节点的值。最小堆则相反。

优先队列可以通过堆实现。各个操作的时间复杂度分别为：- `push`: $O(\log N)$ ；- `pop`: $O(\log N)$ ；- `top`: $O(1)$ ；- 从长度为 N 的数组建堆: $O(N)$ 。

```
class BinaryHeap:    # 在这里我们实现的是最小堆，最大堆几乎是完全一样的
    def __init__(self, nums: list[int] | None = None):
        self._heap = self.buildHeap(nums or [])
        self.size = len(self._heap)
```

```

@staticmethod
# 对堆中的每个非叶子节点倒序执行下降操作
def buildHeap(nums: list[int]) -> list[int]:
    heap = list(nums)
    for idx in range(len(heap) // 2 - 1, -1, -1):
        BinaryHeap._shiftDown(heap, idx, len(heap))
    return heap

def top(self):
    if self.is_empty():
        raise IndexError("top from empty heap")
    return self._heap[0]

def push(self, value):
    self._heap.append(value)
    self.size += 1
    self._shiftUp(self._heap, self.size - 1)

def pop(self):
    if self.is_empty():
        raise IndexError("pop from empty heap")

    self._heap[0], self._heap[self.size - 1] = self._heap[self.size - 1],
    self._heap[0]
    value = self._heap.pop()
    self.size -= 1

    if not self.is_empty():
        self._shiftDown(self._heap, 0, self.size)
    return value

def is_empty(self):
    return self.size == 0

def __len__(self):
    return self.size

@staticmethod
def _shiftUp(nums: list[int], idx: int) -> None:
    while idx > 0:
        parent = (idx - 1) // 2
        if nums[parent] <= nums[idx]:
            break
        nums[parent], nums[idx] = nums[idx], nums[parent]
        idx = parent

@staticmethod
def _shiftDown(nums: list[int], idx: int, size: int) -> None:

```

```
while True:
    left = idx * 2 + 1
    right = idx * 2 + 2
    smallest = idx

    if left < size and nums[left] < nums[smallest]:
        smallest = left
    if right < size and nums[right] < nums[smallest]:
        smallest = right

    if smallest == idx:
        break

    nums[idx], nums[smallest] = nums[smallest], nums[idx]
    idx = smallest
```

4.2 Huffman 树

4.2.1 1. Huffman 树

Huffman 树，又称**最优二叉树**，是一类带权路径长度最小的二叉树。它最常见的用途是构造 Huffman 编码，从而实现对字符序列的压缩。

为了定义 Huffman 树，需要先明确几个概念：

- **权值 (weight)**：节点所代表对象的重要程度。在编码问题中，权值通常是字符出现的频率或次数。
- **路径长度 (path length)**：从根节点到某个节点经过的边数。
- **带权路径长度 (Weighted Path Length, WPL)**：所有叶子节点的权值与其路径长度乘积之和：

$$WPL = \sum_{i=1}^n w_i l_i$$

其中 w_i 表示第 i 个叶子节点的权值， l_i 表示该叶子节点到根节点的路径长度。

Huffman 树解决的问题是：给定 n 个权值，构造一棵有 n 个叶子节点的二叉树，使得整棵树的 WPL 最小。由于权值越大的叶子节点越应该靠近根节点，权值越小的叶子节点越可以放在更深的位置，因此 Huffman 树具有很好的“按频率优化路径长度”的性质。

Huffman 树的构造思想 Huffman 树使用一种典型的**贪心策略**构造：每一步都选择当前权值最小的两棵树合并。具体过程如下：

1. 将每个权值看成一棵只有根节点的二叉树，形成一个森林。
2. 从森林中选出根节点权值最小的两棵树，作为左右子树合并成一棵新树。
3. 新树的根节点权值等于两棵子树根节点权值之和。

4. 将新树放回森林。
5. 重复上述步骤，直到森林中只剩下一棵树，这棵树就是 Huffman 树。

这个算法之所以正确，关键在于：在最优二叉树中，权值最小的两个叶子节点一定可以放在最深层，并且互为兄弟节点。因此每次把最小的两个权值合并，不会破坏整体最优性。

时间复杂度 如果每次都在线性表中寻找两个最小权值，构造复杂度为 $O(n^2)$ 。更常用的做法是使用**最小堆**维护当前森林中的树根，这样每次取出两个最小节点和插入新节点都只需要 $O(\log n)$ ，总时间复杂度为 $O(n \log n)$ 。

```
from heapq import heappop, heappush

class HuffmanNode:
    """Huffman 树节点。"""

    def __init__(self, weight, symbol=None, left=None, right=None):
        self.weight = weight
        self.symbol = symbol
        self.left = left
        self.right = right

    def is_leaf(self):
        return self.left is None and self.right is None

class HuffmanTree:
    """根据符号权值构造 Huffman 树。"""

    def __init__(self, weights):
        if not weights:
            raise ValueError("weights 不能为空")

        for symbol, weight in weights.items():
            if weight <= 0:
                raise ValueError(f"符号 {symbol!r} 的权值必须为正数")

        self.weights = dict(weights)
        self.root = self._build_tree(self.weights)

    def _build_tree(self, weights):
        heap = []
        order = 0

        for symbol, weight in weights.items():
            node = HuffmanNode(weight, symbol=symbol)
            heappush(heap, (weight, order, node))
            order += 1
```

```

    while len(heap) > 1:
        left_weight, _, left = heappop(heap)
        right_weight, _, right = heappop(heap)
        parent = HuffmanNode(left_weight + right_weight, left=left,
right=right)
        heappush(heap, (parent.weight, order, parent))
        order += 1

    return heap[0][2]

def weighted_path_length(self):
    """ 计算 Huffman 树的带权路径长度 WPL。 """

    def dfs(node, depth):
        if node.is_leaf():
            return node.weight * depth
        total = 0
        if node.left is not None:
            total += dfs(node.left, depth + 1)
        if node.right is not None:
            total += dfs(node.right, depth + 1)
        return total

    return dfs(self.root, 0)

def print_tree(self):
    """ 以缩进形式打印 Huffman 树，便于观察构造结果。 """

    def dfs(node, indent=""):
        if node.is_leaf():
            print(f"{indent}{node.symbol!r}: {node.weight}")
            return
        print(f"{indent}*: {node.weight}")
        if node.left is not None:
            dfs(node.left, indent + " ")
        if node.right is not None:
            dfs(node.right, indent + " ")

    dfs(self.root)

# 示例：根据字符出现次数构造 Huffman 树
weights = {"A": 5, "B": 9, "C": 12, "D": 13, "E": 16, "F": 45}
tree = HuffmanTree(weights)
tree.print_tree()
print("WPL =", tree.weighted_path_length())

```

输出：

```
*: 100
  'F': 45
*: 55
  *: 25
    'C': 12
    'D': 13
  *: 30
    *: 14
      'A': 5
      'B': 9
    'E': 16
WPL = 224
```

4.2.2 2. Huffman 编码

Huffman 编码是一种**变长前缀编码**。它利用 Huffman 树为每个符号分配一个二进制编码：

- 从根节点出发，走向左子树记为 0；
- 走向右子树记为 1；
- 从根节点到某个叶子节点经过的 0、1 序列，就是该叶子节点对应符号的编码。

由于 Huffman 树中的每个符号都存储在叶子节点上，任何一个符号的编码都不可能是另一个符号编码的前缀。因此 Huffman 编码是一种**前缀码**。前缀码的好处是解码时不需要分隔符：从根节点开始，根据比特串逐位向左或向右走，只要走到叶子节点，就能确定一个完整符号，然后回到根节点继续解码。

为什么 Huffman 编码可以压缩数据 固定长度编码会给每个符号分配相同长度的二进制串。例如有 8 种字符时，每个字符至少需要 3 位。但在真实文本中，不同字符出现频率往往差异很大。Huffman 编码让出现频率高的字符拥有更短的编码，让出现频率低的字符使用更长的编码，从而降低整体平均编码长度。

例如，如果字符 F 的出现次数远高于其他字符，它在 Huffman 树中通常会更靠近根节点，对应编码也更短。这样在编码长文本时，频繁出现的字符会节省大量空间。

编码与解码过程 编码过程：

1. 统计文本中每个符号的出现次数。
2. 根据频率构造 Huffman 树。
3. 遍历 Huffman 树，得到每个符号对应的二进制编码表。
4. 将原文本中的每个符号替换成对应编码，得到完整比特串。

解码过程：

1. 从 Huffman 树的根节点开始。
2. 读取比特串中的下一位：读到 0 走左子树，读到 1 走右子树。
3. 如果到达叶子节点，就输出该叶子节点对应的符号，并回到根节点。
4. 重复直到比特串读取完毕。

需要注意的是，解码必须使用与编码时相同的 Huffman 树，或者至少使用同一张编码表。否则相同的比特串可能被解释成不同的文本。

```
class HuffmanCodec:
    """Huffman 编码器与解码器。"""

    def __init__(self, text):
        if not text:
            raise ValueError("text 不能为空")

        self.frequencies = {}
        for ch in text:
            self.frequencies[ch] = self.frequencies.get(ch, 0) + 1
        self.tree = HuffmanTree(self.frequencies)
        self.codes = self._build_codes()
        self.reverse_codes = {code: symbol for symbol, code in self.codes.items()}

    def _build_codes(self):
        codes = {}

        def dfs(node, prefix):
            if node.is_leaf():
                # 只有一种字符时，给它分配编码 0。
                codes[node.symbol] = prefix or "0"
                return
            if node.left is not None:
                dfs(node.left, prefix + "0")
            if node.right is not None:
                dfs(node.right, prefix + "1")

        dfs(self.tree.root, "")
        return codes

    def encode(self, text):
        """将文本编码为 0/1 组成的字符串。"""

        try:
            return "".join(self.codes[ch] for ch in text)
        except KeyError as exc:
            raise ValueError(f"字符 {exc.args[0]!r} 不在当前 Huffman 编码表中")
            from exc

    def decode(self, bits):
        """将 0/1 字符串解码为原文本。"""

        if any(bit not in "01" for bit in bits):
            raise ValueError("bits 只能包含字符 '0' 和 '1'")

        # 单字符文本的 Huffman 树只有一个节点，需要单独处理。
        if self.tree.root.is_leaf():
            return self.tree.root.symbol
```



```

        if any(bit != "0" for bit in bits):
            raise ValueError("bits 不是当前 Huffman 树生成的合法编码")
        return self.tree.root.symbol * len(bits)

result = []
node = self.tree.root

for bit in bits:
    node = node.left if bit == "0" else node.right
    if node is None:
        raise ValueError("bits 不是当前 Huffman 树生成的合法编码")
    if node.is_leaf():
        result.append(node.symbol)
        node = self.tree.root

if node is not self.tree.root:
    raise ValueError("bits 在某个字符编码中间结束, 无法完整解码")

return "".join(result)

def compression_info(self, text):
    """ 比较 Huffman 编码和定长编码的大致位数。 """

    encoded_length = len(self.encode(text))
    alphabet_size = len(self.codes)
    fixed_width = max(1, (alphabet_size - 1).bit_length())
    fixed_length = len(text) * fixed_width
    return {
        "alphabet_size": alphabet_size,
        "fixed_width": fixed_width,
        "fixed_length": fixed_length,
        "huffman_length": encoded_length,
        "saved_bits": fixed_length - encoded_length,
    }

# 示例: 编码与解码
text = "this is an example for huffman encoding"
codec = HuffmanCodec(text)
encoded = codec.encode(text)
decoded = codec.decode(encoded)

print(" 编码表:")
for ch, code in sorted(codec.codes.items(), key=lambda item: (len(item[1]),
item[0])):
    display_ch = " 空格" if ch == " " else ch
    print(f"{{display_ch!r}}: {{code}}")

print("\n\n原文本:", text)

```

```
print(" 编码结果:", encoded)
print(" 解码结果:", decoded)
print(" 解码正确:", decoded == text)
print(" 压缩信息:", codec.compression_info(text))
```

输出:

编码表:

```
'空格': 101
'n': 000
'a': 1100
'e': 1101
'f': 1110
'i': 1001
'm': 0011
'o': 0100
's': 0010
'c': 10000
'd': 10001
'g': 11110
'h': 11111
'l': 01101
'p': 01100
'r': 01110
't': 01010
'u': 01111
'x': 01011
```

原文本: this is an example for huffman encoding

编码结果:

```
0101011111100100101011001001010111000001011101010111100001101
100011011101101111001000111010111111011111101111101110001111000001
01110100010000010010001100100011110
```

解码结果: this is an example for huffman encoding

解码正确: True

```
压缩信息: {'alphabet_size': 19, 'fixed_width': 5, 'fixed_length': 195,
'huffman_length': 157, 'saved_bits': 38}
```

4.3 二叉查找树与 AVL 树

二叉查找树 (Binary Search Tree, 简称 BST) 是一种非常经典的数据结构, 它结合了二叉树和有序查找的特点, 在查找、插入和删除操作上具有较高效率。二叉查找树满足以下性质: 对于树中的任意一个节点:

- 左子树上所有节点的值都小于当前节点的值;
- 右子树上所有节点的值都大于当前节点的值;
- 左右子树本身也必须分别是二叉查找树。

根据定义可知, 二叉查找树的中序遍历 (DFS) 序列一定是从小到大排列的。

设二叉查找树的树高为 h ，那么查找、删除、插入三个基本操作的时间复杂度均为 $O(h)$ 。特别的，如果二叉查找树是平衡的，即 $h \approx \log_2 N$ ，那么查找、删除、插入的时间复杂度为 $O(\log N)$ 。

下面我们给出二叉查找树的具体实现。

```
class BSTNode:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None

class BinarySearchTree:
    def __init__(self):
        self.root = None

    def insert(self, value):
        self.root = self._insert(self.root, value)

    def _insert(self, node, value):
        if node is None:
            return BSTNode(value)

        if value < node.value:
            node.left = self._insert(node.left, value)
        elif value > node.value:
            node.right = self._insert(node.right, value)

        return node

    def search(self, value):
        return self._search(self.root, value)

    def _search(self, node, value):
        if node is None or node.value == value:
            return node
        if value < node.value:
            return self._search(node.left, value)
        return self._search(node.right, value)

    def delete(self, value):
        self.root = self._delete(self.root, value)

    def _delete(self, node, value):
        if node is None:
            return None

        if value < node.value:
            node.left = self._delete(node.left, value)
        elif value > node.value:
            node.right = self._delete(node.right, value)
```

```

    else:
        if node.left is None:
            return node.right
        if node.right is None:
            return node.left

        successor = self._find_min_node(node.right)
        node.value = successor.value
        node.right = self._delete(node.right, successor.value)

    return node

def _find_min_node(self, node):
    cur = node
    while cur.left is not None:
        cur = cur.left
    return cur

def inOrderTraversal(self):
    res = []

    def _inOrder(node):
        if node is None:
            return
        _inOrder(node.left)
        res.append(node.value)
        _inOrder(node.right)

    _inOrder(self.root)
    return res

```

4.3.1 AVL 树

上文提到，如果二叉搜索树是平衡的，即树的高度大约是 $\log_2 N$ ，那么插入、删除、搜索三个基本操作都可在对数复杂度内完成。但是，如果搜索树严重不平衡，那么三个操作的时间复杂度就会严重退化。例如，如果对一个从小到大排列的列表依次进行插入操作建立二叉搜索树，那么得到的树就是完全右偏的树——每个节点要么是叶节点，要么只有右子节点。此时树高为 $O(N)$ ，三种基本操作的时间复杂度也全部退化到 $O(N)$ 。

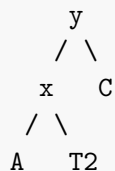
要防止出现上述性能退化，关键在于二叉搜索树应当是接近于“平衡”的，左右子树的高度不应相差太大。这就引出了 AVL 树。

我们首先给出平衡因子的定义：对于一个节点，其平衡因子定义为左子树高度减去右子树高度。如果平衡因子为 0，那么称这个节点是平衡的。如果平衡因子大于 0，那么节点的左子树比右子树高，称之为“左重”。如果平衡因子小于 0，那么节点的右子树比左子树高，称之为“右重”。

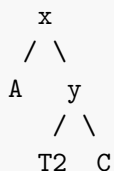
AVL 树是一棵满足以下条件的二叉搜索树：对于任意一个节点，它的平衡因子绝对值不超过 1。

AVL 树的旋转与再平衡 AVL 树的核心思想是：每次插入或删除后，沿着递归路径从下往上检查节点高度。一旦某个节点的左右子树高度差超过 1，就通过局部旋转把这棵子树重新调整平衡。旋转只改变少数几个指针，不会破坏二叉搜索树的大小关系，因此中序遍历结果仍然有序。

右旋 右旋用于处理“左边太高”的情况。设当前失衡节点为 y ，它的左孩子为 x ， x 的右子树为 $T2$ 。右旋前的局部结构如下：



右旋时，让 x 上升成为这棵子树的新根，让 y 下降成为 x 的右孩子。原来夹在 x 和 y 之间的 $T2$ ，需要接到 $y.left$ 上：



对应到代码，就是：

```

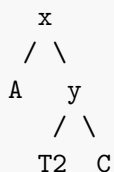
x = y.left
t2 = x.right

x.right = y
y.left = t2

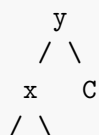
```

为什么这样仍然是二叉搜索树？因为旋转前有 $A < x < T2 < y < C$ ，旋转后这个相对顺序没有改变，只是根节点从 y 变成了 x 。最后要先更新下降节点 y 的高度，再更新上升节点 x 的高度，并返回新的子树根 x 。

左旋 左旋用于处理“右边太高”的情况，过程与右旋对称。设当前失衡节点为 x ，它的右孩子为 y ， y 的左子树为 $T2$ 。左旋前：



左旋后：



A T2

对应到代码，就是：

```
y = x.right
t2 = y.left

y.left = x
x.right = t2
```

同理，旋转前后保持 $A < x < T2 < y < C$ ，所以二叉搜索树性质不会被破坏。最后要先更新下降节点 x 的高度，再更新上升节点 y 的高度，并返回新的子树根 y 。

四种失衡类型 AVL 树的失衡可以分成四类。判断时先看当前节点 `node` 的平衡因子，再看较高子树根节点的平衡因子。

- LL 型：`node` 左重，并且 `node.left` 也左重或平衡。说明新变化发生在左子树的左侧，直接对 `node` 做一次右旋。
- LR 型：`node` 左重，但 `node.left` 右重。说明局部结构是“折线”，先对 `node.left` 做左旋，把它转成 LL 型，再对 `node` 做右旋。
- RR 型：`node` 右重，并且 `node.right` 也右重或平衡。说明新变化发生在右子树的右侧，直接对 `node` 做一次左旋。
- RL 型：`node` 右重，但 `node.right` 左重。说明局部结构是“折线”，先对 `node.right` 做右旋，把它转成 RR 型，再对 `node` 做左旋。

也就是说，LL 和 RR 是“直线型”失衡，只需要一次旋转；LR 和 RL 是“折线型”失衡，需要先旋转子节点，再旋转当前节点。

rebalance 的流程 `_rebalance(node)` 的工作可以概括为三步：

1. 先调用 `_update_height(node)`，因为插入或删除后，当前节点的子树高度可能已经变化。
2. 计算当前节点的平衡因子 `balance = _get_balance(node)`。
3. 根据 `balance` 和子节点的平衡因子判断属于 LL、LR、RR、RL 中的哪一种情况，并返回旋转后的新子树根。

对应到代码逻辑是：

```
if balance > 1 and self._get_balance(node.left) >= 0:
    return self._right_rotate(node)          # LL

if balance > 1 and self._get_balance(node.left) < 0:
    node.left = self._left_rotate(node.left)
    return self._right_rotate(node)          # LR

if balance < -1 and self._get_balance(node.right) <= 0:
    return self._left_rotate(node)           # RR

if balance < -1 and self._get_balance(node.right) > 0:
```

```

        node.right = self._right_rotate(node.right)
        return self._left_rotate(node)          # RL

return node

```

这里必须返回新的子树根，因为旋转后局部根节点可能已经改变。递归调用的上一层会用 `node.left = ...` 或 `node.right = ...` 接住这个返回值；如果旋转发生在整棵树的根节点处，外层的 `insert` 或 `delete` 会把 `self.root` 更新为新的根。

AVL 代码实现思路 实现 AVL 树时，可以在普通 BST 的基础上增加高度维护和再平衡逻辑：

- `AVLNode` 继承 `BSTNode`，额外保存 `height`。空节点高度记为 0，叶节点高度记为 1。
- `_height(node)` 统一处理空节点，避免每次计算高度前都判断 `None`。
- `_update_height(node)` 用左右子树高度更新当前节点高度，公式为 $1 + \max(\text{left_height}, \text{right_height})$ 。
- `_get_balance(node)` 返回左子树高度减右子树高度，用来判断是否失衡以及失衡方向。
- `_left_rotate` 和 `_right_rotate` 只负责局部指针调整、更新高度、返回新的子树根。
- `_rebalance` 负责统一判断四种失衡类型，并调用对应的旋转操作。
- `_insert` 仍然先按 BST 规则递归插入；递归返回时，对路径上的每个祖先节点执行 `_rebalance`。
- `_delete` 也先按 BST 规则删除节点；如果被删节点有两个孩子，就用右子树中的最小节点作为后继替换当前值，再删除后继节点；递归返回时同样执行 `_rebalance`。

插入和删除都要在递归返回阶段做再平衡，这是因为真正发生高度变化的位置在更深层。只有从下往上更新高度并检查平衡因子，才能保证整条路径上的所有祖先节点重新满足 AVL 条件。

```

class AVLNode(BSTNode):
    def __init__(self, value):
        super().__init__(value)
        self.height = 1

class AVLTree(BinarySearchTree):
    def _height(self, node):
        if node is None:
            return 0
        return node.height

    def _update_height(self, node):
        node.height = 1 + max(
            self._height(node.left),
            self._height(node.right)
        )

```

```
def _get_balance(self, node):
    if node is None:
        return 0
    return self._height(node.left) - self._height(node.right)

def _right_rotate(self, y):
    x = y.left
    t2 = x.right

    # 执行右旋
    x.right = y
    y.left = t2

    # 先更新 y, 再更新 x
    self._update_height(y)
    self._update_height(x)

    return x

def _left_rotate(self, x):
    y = x.right
    t2 = y.left

    # 执行左旋
    y.left = x
    x.right = t2

    # 先更新 x, 再更新 y
    self._update_height(x)
    self._update_height(y)

    return y

def _rebalance(self, node):
    self._update_height(node)

    balance = self._get_balance(node)

    # LL 型: 左子树的左侧过高
    if balance > 1 and self._get_balance(node.left) >= 0:
        return self._right_rotate(node)

    # LR 型: 左子树的右侧过高
    if balance > 1 and self._get_balance(node.left) < 0:
        node.left = self._left_rotate(node.left)
        return self._right_rotate(node)

    # RR 型: 右子树的右侧过高
    if balance < -1 and self._get_balance(node.right) <= 0:
```



```
        return self._left_rotate(node)

    # RL 型：右子树的左侧过高
    if balance < -1 and self._get_balance(node.right) > 0:
        node.right = self._right_rotate(node.right)
        return self._left_rotate(node)

    return node

def _insert(self, node, value):
    if node is None:
        return AVLNode(value)

    if value < node.value:
        node.left = self._insert(node.left, value)
    elif value > node.value:
        node.right = self._insert(node.right, value)
    else:
        # 不插入重复值
        return node

    return self._rebalance(node)

def _delete(self, node, value):
    if node is None:
        return None

    if value < node.value:
        node.left = self._delete(node.left, value)

    elif value > node.value:
        node.right = self._delete(node.right, value)

    else:
        # 情况 1: 没有左孩子
        if node.left is None:
            return node.right

        # 情况 2: 没有右孩子
        if node.right is None:
            return node.left

        # 情况 3: 左右孩子都有
        successor = self._find_min_node(node.right)
        node.value = successor.value
        node.right = self._delete(node.right, successor.value)

    return self._rebalance(node)
```

5 排序、查找与字符串匹配

5.1 排序算法

在本 notebook 中，我们将分别介绍以下几种经典的排序算法：冒泡排序、选择排序、插入排序、希尔排序、归并排序以及快速排序。我们将分别介绍这些排序算法的思想并实现，同时，我们也会具体分析它们的算法复杂度。

5.1.1 冒泡排序

冒泡排序的核心思想是：反复比较相邻的两个元素，如果顺序不对就交换。每一轮都会把当前最大的元素“冒泡”到最后。

冒泡排序的空间复杂度为 $O(1)$ 。在不做优化的情况下，冒泡排序的时间复杂度为 $O(N^2)$ 。如果加入判断，当数组已经有序时就不再进行冒泡，此时时间复杂度可以得到优化：当数组已经有序时，时间复杂度为 $O(N)$ 。此时最坏时间复杂度、平均时间复杂度仍为 $O(N^2)$ 。

冒泡排序是稳定的。

```
def bubbleSort(nums: list[int]) -> None:
    for i in range(len(nums)):
        for j in range(0, len(nums) - i - 1):
            if nums[j] > nums[j + 1]:
                nums[j], nums[j + 1] = nums[j + 1], nums[j]

def modifiedBubbleSort(nums: list[int]) -> None:
    for i in range(len(nums)):
        swapped = False
        for j in range(0, len(nums) - i - 1):
            if nums[j] > nums[j + 1]:
                nums[j], nums[j + 1] = nums[j + 1], nums[j]
                swapped = True
        if not swapped:
            break

nums = [5, 1, 3, 6, 4]
bubbleSort(nums)
print(nums)
```

输出：

```
[1, 3, 4, 5, 6]
```

5.1.2 选择排序

选择排序的核心思想是：每一轮从未排序部分中找出最小的元素，放到当前未排序部分的最前面。

选择排序的空间复杂度为 $O(1)$ 。时间复杂度为 $O(N^2)$ 。

注意，选择排序是不稳定的。原因是：它每一轮会把“未排序部分的最小值”和当前位置的元素直接交换，这个交换可能会把某个相同元素的相对顺序打乱。以下是一个具体的反例：

```
[2a, 2b, 1]
```

经过一次选择交换后变为

```
[1, 2b, 2a]
```

此时列表已经有序，但是两个 2 之间的相对位置已经被打乱了。

```
def selectionSort(nums: list[int]) -> None:
    for i in range(len(nums) - 1):
        idx_of_smallest = i

        for j in range(i, len(nums)):
            if nums[j] < nums[idx_of_smallest]:
                idx_of_smallest = j

        nums[i], nums[idx_of_smallest] = nums[idx_of_smallest], nums[i]

nums = [5, 1, 3, 6, 4]
selectionSort(nums)
print(nums)
```

输出：

```
[1, 3, 4, 5, 6]
```

5.1.3 插入排序

插入排序的核心思想与我们平时玩纸牌时对纸牌排序的思路是一致的：维持一个已经排好序的子列表，将这个列表置于整个列表的前端；依次将子列表之外的元素插入到子列表中，逐渐扩大有序的子列表，直至所有元素都被扩展到子列表中。

由于不需要额外的空间，插入排序的空间复杂度是 $O(1)$ 。在最坏的情况下，即列表逆序的时候，每个元素在插入到子列表的过程中，需要与其中的每个元素发生换序，因此最坏时间复杂度是 $O(N^2)$ 。相反地，如果列表已经排好了序，那么时间复杂度降为 $O(N)$ 。平均情况下，时间复杂度为 $O(N^2)$ 。

插入排序是稳定的。

```
def insertionSort(nums: list[int]) -> None:
    for i in range(1, len(nums)):
```

```

    for j in reversed(range(i)):
        if nums[j + 1] < nums[j]:
            nums[j + 1], nums[j] = nums[j], nums[j + 1]
        else:
            break

nums = [5, 1, 3, 6, 4]
insertionSort(nums)
print(nums)

# 当然，还可以做进一步的优化
def modifiedInsertionSort(nums: list[int]) -> None:
    for i in range(1, len(nums)):
        cur = nums[i]
        j = i

        while j >= 1 and nums[j - 1] > cur:
            nums[j] = nums[j - 1]
            j -= 1

        nums[j] = cur

nums = [5, 1, 3, 6, 4]
modifiedInsertionSort(nums)
print(nums)

```

输出：

```

[1, 3, 4, 5, 6]
[1, 3, 4, 5, 6]

```

5.1.4 希尔排序

希尔排序可以理解为：插入排序的升级版。

普通插入排序每次只能把元素一点点往前移动，遇到很小的元素在数组后面时，需要移动很多次。希尔排序的思路是：先让元素“大步移动”，再逐渐缩小步长，最后变成普通插入排序。

具体而言，我们会预先设定一组递减到 1 的 gaps 序列。对于 gap 序列中的每一个 gap 值，我们将待排序数组切分为若干个子数组，其中每段在原始数组中间隔为 gap。然后在每一个子数组内分别进行插入排序。当 gap 取 1 时，这时的插入排序相当于普通的插入排序，但是，由于数组已经大致有序了，此时再进行插入排序会快很多。

希尔排序的空间复杂度为 $O(1)$ 。希尔排序的时间复杂度依赖于 gap 序列的选取，通常认为其时间复杂度介于 $O(N^{1.3}) \sim O(N^2)$ 之间，最坏情况下退化为 $O(N^2)$ 。

希尔排序是不稳定的排序。

```
def shellSort(nums: list[int], gaps: list[int] | None = None) -> None:
    if gaps is None:
        gaps = []
        n = len(nums) // 2
        while n > 0:
            gaps.append(n)
            n //= 2

    for gap in gaps:
        for i in range(gap, len(nums)):
            cur = nums[i]
            j = i

            while j >= gap and nums[j - gap] > cur:
                nums[j] = nums[j - gap]
                j -= gap

            nums[j] = cur

nums = [5, 1, 3, 6, 9]
shellSort(nums)
print(nums)
```

输出:

```
[1, 3, 5, 6, 9]
```

5.1.5 归并排序

归并排序是一种经典的分治排序算法。它的核心思想是：把一个大数组不断拆成更小的子数组，直到每个子数组只剩一个元素；然后再把这些有序的小数组逐步合并成更大的有序数组。

归并排序的核心在于将两个有序的小数组合并为一个更大的有序数组——而这是可以通过双指针得到简单的实现。

归并排序的时间复杂度（最优/最差/平均）为 $O(N \log N)$ ：无论数组是否有序，数组都会经过大约 $\log N$ 次切分，而每次切分后合并两个有序数组需要经过 $O(N)$ 次移动，故总的时间复杂度为 $O(N \log N)$ 。

归并排序的实现需要利用到 $O(N)$ 长度的辅助列表，因此其空间复杂度为 $O(N)$ 。

归并排序是稳定的。

```
# 注意，该版本的归并排序中的切片操作会导致额外的空间消耗，使得空间复杂度超过  $O(N)$ 
def mergeSort(nums: list[int]) -> list[int]:
    if len(nums) <= 1:
        return nums

    n = len(nums) // 2
```

```

    left = mergeSort(nums[:n])
    right = mergeSort(nums[n:])

    return _merge(left, right)

def _merge(left: list[int], right: list[int]) -> list[int]:
    result = []
    i, j = 0, 0
    len_left, len_right = len(left), len(right)

    while i < len_left and j < len_right:
        if left[i] <= right[j]:
            result.append(left[i])
            i += 1
        else:
            result.append(right[j])
            j += 1

    while i < len_left:
        result.append(left[i])
        i += 1

    while j < len_right:
        result.append(right[j])
        j += 1

    return result

nums = [5, 1, 3, 6, 9]
nums = mergeSort(nums)
print(nums)

```

输出:

```
[1, 3, 5, 6, 9]
```

```

# 优化版本的归并排序通过一个辅助列表实现  $O(N)$  的空间复杂度
def modifiedMergeSort(nums: list[int]) -> None:
    if len(nums) <= 1:
        return

    temp = nums[:]
    _modifiedMergeSort(nums, 0, len(nums) - 1, temp)

def _modifiedMergeSort(nums: list[int], left: int, right: int, temp: list[int]) ->
None:

```

```
if left >= right:
    return

mid = (left + right) // 2

_modifiedMergeSort(nums, left, mid, temp)
_modifiedMergeSort(nums, mid + 1, right, temp)

# 如果左右两部分已经整体有序，直接跳过合并
if nums[mid] <= nums[mid + 1]:
    return

_mergeWithTemp(nums, left, mid, right, temp)

def _mergeWithTemp(nums: list[int], left: int, mid: int, right: int, temp:
list[int]) -> None:
    for p in range(left, right + 1):
        temp[p] = nums[p]

    i, j = left, mid + 1

    for k in range(left, right + 1):
        if i > mid:
            nums[k] = temp[j]
            j += 1
        elif j > right:
            nums[k] = temp[i]
            i += 1
        elif temp[i] <= temp[j]:
            nums[k] = temp[i]
            i += 1
        else:
            nums[k] = temp[j]
            j += 1

nums = [5, 1, 3, 6, 9]
modifiedMergeSort(nums)
print(nums)
```

输出：

```
[1, 3, 5, 6, 9]
```

5.1.6 快速排序

快速排序是一种经典的分治排序算法。它的核心思想是：选一个元素作为“基准值”pivot，把数组分成两部分：小于基准值的放左边，大于基准值的放右边，然后分别

对左右两部分继续排序。

快速排序的性能取决于 `pivot` 的选取：如果 `pivot` 差不多位于数组中间时，它能够将数组平均切分为两半。此时的最优时间复杂度为 $O(N \log N)$ 。但是，如果 `pivot` 接近数组的最值，切分操作就白做了，因此最坏时间复杂度通常会退化到 $O(N^2)$ 。快速排序的平均复杂度为 $O(N \log N)$ 。

快速排序的空间复杂度取决于调用栈的深度。最优能达到 $O(\log N)$ ，最差情况退化到 $O(N)$ 。

快速排序通常是非稳定的排序。

```
# 以下的实现中使用了额外的数组，空间复杂度超过  $O(\log N)$ 
def quickSort(nums: list[int]) -> list[int]:
    if len(nums) <= 1:
        return nums

    pivot_idx = len(nums) // 2
    pivot = nums[pivot_idx]

    left, right = [], []

    for i in range(len(nums)):
        if i != pivot_idx:
            if nums[i] < pivot:
                left.append(nums[i])
            else:
                right.append(nums[i])

    return quickSort(left) + [pivot] + quickSort(right)

nums = [5, 1, 3, 6, 9]
print(quickSort(nums))
```

输出：

```
[1, 3, 5, 6, 9]
```

```
# 原地排序的优化版快速排序可以将平均空间复杂度降低到  $O(\log N)$ 
def modifiedQuickSort(nums: list[int]) -> None:
    _modifiedQuickSort(nums, 0, len(nums) - 1)

def _modifiedQuickSort(nums: list[int], left: int, right: int) -> None:
    if left >= right:
        return

    pivot_idx = _partition(nums, left, right)

    _modifiedQuickSort(nums, left, pivot_idx - 1)
    _modifiedQuickSort(nums, pivot_idx + 1, right)
```



```
def _partition(nums: list[int], left: int, right: int) -> int:
    pivot = nums[right]
    i = left - 1

    for j in range(left, right):
        if nums[j] <= pivot:
            i += 1
            nums[i], nums[j] = nums[j], nums[i]

    nums[i + 1], nums[right] = nums[right], nums[i + 1]

    return i + 1

nums = [6, 3, 8, 5, 2, 7, 4]
modifiedQuickSort(nums)
print(nums)
```

输出:

```
[2, 3, 4, 5, 6, 7, 8]
```

5.2 二分查找

二分查找是一种用于在有序列表中查找特定数据项的搜索算法。在每次搜索中，列表从中间项被一分为二，由于列表是有序的，因此我们只需要对比待查找数据项与列表中间值的大小，就可以知道待查找数据项在前半列表或是后半列表中。以此类推，每次我们都将剩余的列表从中间一分为二，重复这一过程，我们要么在某一步查找到数据项，要么直至列表不可再被分割也没有查找到数据项，此时我们断定列表中没有待查找的数据项。

具体而言，以下几个搜索问题是二分查找的典型应用场景。

5.2.1 Problem 1

在排好序的数组中查找 `target`。注意，如果 `target` 有重复，普通二分查找只能找到其中一个。

```
from typing import List, Optional

def find_target(nums: List[int], target: int) -> Optional[int]:
    # 假设数组是从小到大排列的
    left, right = 0, len(nums) - 1

    while left <= right:
        mid = left + (right - left) // 2
```

```

    if nums[mid] == target:
        return mid
    elif nums[mid] < target:
        left = mid + 1
    else:
        right = mid - 1

return None

```

5.2.2 Problem 2

寻找满足条件 `check(item)=True` 的第一个项。其中条件 `check` 具有以下性质：若待查找列表的某一项满足该条件，那么列表后面的所有项都满足这一条件。也就是说，列表按 `check` 结果看应当呈现 `False, False, ..., True, True, ...` 的单调结构。

因此，假设待查找项的角标为 `k`，那么 `k` 满足：- `check(items[k-1])=False` - `check(items[k])=True`

```

# 我们首先定义条件 check 函数
def check_positive(item) -> bool:
    # 假设条件为 item 为正数
    return item > 0

def find_first(items: List[int]) -> Optional[int]:
    if not items:
        return None

    left, right = 0, len(items) - 1

    while left < right:
        mid = left + (right - left) // 2
        if check_positive(items[mid]):
            right = mid
        else:
            left = mid + 1

    return left if check_positive(items[left]) else None

```

5.2.3 Problem 3

寻找满足条件 `check(item)=True` 的最后一个项。其中条件 `check` 具有以下性质：若待查找列表的某一项满足该条件，那么列表前面的所有项都满足这一条件。也就是说，列表按 `check` 结果看应当呈现 `True, True, ..., False, False, ...` 的单调结构。

因此，假设待查找项的角标为 `k`，那么 `k` 满足：- `check(items[k+1])=False` - `check(items[k])=True`

```
def check_negative(item) -> bool:
    # 假设条件为 item 为负数
    return item < 0

def find_last(items: List[int]) -> Optional[int]:
    if not items:
        return None

    left, right = 0, len(items) - 1

    while left < right:
        mid = left + (right - left + 1) // 2
        if check_negative(items[mid]):
            left = mid
        else:
            right = mid - 1

    return left if check_negative(items[left]) else None
```

5.2.4 更多应用场景

二分查找的核心不只是“在有序数组里找某个值”，而是在一个有单调性的搜索空间里找边界。

- 查找插入位置：例如 Python 标准库 `bisect_left` / `bisect_right`，可以确定新元素应插入到哪里，或统计某个值出现了多少次。
- 查找第一个不小于目标值的位置：也叫 lower bound，例如在分数线、时间戳、版本号中找第一个达到要求的位置。
- 查找最后一个不大于目标值的位置：也叫 upper bound 的相关变形，例如在日志中找不晚于某个时间的最后一条记录。
- 二分答案：答案本身不是数组元素，但可以判断某个候选答案是否可行。例如最小化最大负载、求最少天数、求最大可行长度、运输能力问题等。
- 连续区间二分：在实数范围内逼近满足条件的值，例如求平方根、求方程近似解。
- 在旋转有序数组、二维有序矩阵中查找：这些问题不是最基础形式，但仍利用有序性逐步缩小搜索范围。

5.3 KMP 字符串匹配算法

KMP (Knuth-Morris-Pratt) 用于在主串 `text` 中查找模式串 `pattern` 出现的位置。

它解决的问题是：当匹配到一半发生失配时，朴素算法会反复比较已经确认过的字符；KMP 利用模式串自身的结构，避免主串指针回退。

5.3.1 1. KMP 的动机

先看朴素字符串匹配。假设要在 `text` 中查找 `pattern`，朴素做法会枚举每一个可能的起点 `start`，然后从头比较模式串：

```
text:    a a a a a b
pattern: a a a b
         ^ ^ ^ x
```

在起点 0，前三个字符都匹配，最后一个字符失配。朴素算法会把起点右移一位，然后重新从 `pattern[0]` 开始比较：

```
text:    a a a a a b
pattern:  a a a b
         ^ ^ ^ x
```

问题在于：前面已经比较过很多 `a`，但朴素算法没有利用这些信息。

如果 `text = "aaaaaaaaab"`，`pattern = "aaaab"`，很多起点都会先成功匹配多个 `a`，再在 `b` 处失败。因此最坏时间复杂度是：

$$O(nm)$$

其中 $n = \text{len}(\text{text})$ ， $m = \text{len}(\text{pattern})$ 。

KMP 的核心目标是：

主串指针 `i` 永不回退；失配时只移动模式串指针 `j`。

朴素匹配实现 下面的函数返回 `pattern` 在 `text` 中出现的所有起始下标。

```
def naive_search(text: str, pattern: str) -> list[int]:
    """ 返回 pattern 在 text 中出现的所有起始下标。 """
    n, m = len(text), len(pattern)

    if m == 0:
        return list(range(n + 1))

    matches = []

    for start in range(n - m + 1):
        j = 0
        while j < m and text[start + j] == pattern[j]:
            j += 1

        if j == m:
            matches.append(start)

    return matches
```

```
naive_search("ABABDABACDABABCABAB", "ABABCABAB")
```

5.3.2 2. LPS 数组

KMP 的关键预处理结构是 LPS 数组。

LPS 是 **Longest Proper Prefix which is also Suffix** 的缩写。

对模式串 `pattern` 的每一个位置 `i`, `lps[i]` 表示:

在子串 `pattern[0:i+1]` 中, 最长的、既是前缀又是后缀的真前缀长度。

这里的“真前缀”表示不能等于字符串本身。

例如:

字符串	最长相等的真前缀和后缀	LPS 值
a	无	0
ab	无	0
aba	a	1
abab	ab	2
ababa	aba	3

所以对 `pattern = "ababaca"`:

i	pattern[0:i+1]	lps[i]
0	a	0
1	ab	0
2	aba	1
3	abab	2
4	ababa	3
5	ababac	0
6	ababaca	1

因此:

```
build_lps("ababaca") == [0, 0, 1, 2, 3, 0, 1]
```

假设匹配过程中已经有一段成功匹配:

```
text:    ... a b a b a x ...
pattern:  a b a b a c
          0 1 2 3 4 5
```

前 5 个字符 `ababa` 匹配成功, 但在下一个字符处失配: `x != c`。

这时不需要把模式串完全移到下一位重新比较, 因为已匹配片段 `ababa` 内部有结构:

```
ababa
前缀: aba
后缀: aba
```

也就是说，已经匹配上的后缀 `aba` 可以继续作为下一轮匹配的前缀。模式串指针 `j` 可以从 5 回退到 3，不需要让主串指针 `i` 回退。

因此，`lps[j - 1]` 的含义正好是：

当前已经匹配 `j` 个字符，下一位失配时，模式串指针 `j` 应该回退到哪里。

这就是 KMP 不重复比较主串字符的根本原因。

接下来我们考虑 `lps` 数组的构造。

现在要对模式串本身做一次“自匹配”。

维护两个变量：

- `i`：当前正在计算 `lps[i]`。
- `length`：当前候选的最长相等前后缀长度，也可以理解为“前缀指针”。

在计算 `lps[i]` 时：

1. 如果 `pattern[i] == pattern[length]`，说明当前相等前后缀可以延长一位：

```
length += 1
lps[i] = length
i += 1
```

2. 如果 `pattern[i] != pattern[length]`，但 `length > 0`，说明当前候选长度太长，需要缩短候选前缀。新的候选长度不是简单地 `length - 1`，而是：

```
length = lps[length - 1]
```

原因是：如果长度为 `length` 的前缀无法继续匹配，那么下一个可能的候选，应该是这个前缀自身的最长相等真前后缀。

3. 如果 `pattern[i] != pattern[length]` 且 `length == 0`，说明没有更短的候选了：

```
lps[i] = 0
i += 1
```

注意第 2 种情况中，`i` 不移动。因为只是更换候选前缀，当前字符 `pattern[i]` 还没有被处理完。

构造过程的时间复杂度是 $O(m)$ 。虽然 `length` 有时会回退，但每次回退都对应之前已经增长过的长度，总次数受 `m` 限制。

```
def build_lps(pattern: str) -> list[int]:
    """ 构造 pattern 的 LPS 数组。 """
    m = len(pattern)
    lps = [0] * m

    length = 0 # 当前候选的最长相等前后缀长度
```

```

i = 1          # lps[0] 一定是 0, 因此从 1 开始

while i < m:
    if pattern[i] == pattern[length]:
        length += 1
        lps[i] = length
        i += 1
    elif length > 0:
        length = lps[length - 1]
    else:
        lps[i] = 0
        i += 1

return lps

```

```
build_lps("ababaca")
```

LPS 构造实现

5.3.3 3. 最终 KMP 算法

KMP 搜索阶段维护两个指针：

- i ：主串 `text` 的当前位置。
- j ：模式串 `pattern` 的当前位置，也表示当前已经匹配的模式串长度。

匹配规则：

1. 如果 `text[i] == pattern[j]`，两个指针都向右移动。
2. 如果 `j == len(pattern)`，说明找到一次完整匹配，起点是 $i - j$ 。为了继续寻找重叠匹配，把 j 回退到 `lps[j - 1]`。
3. 如果 `text[i] != pattern[j]` 且 $j > 0$ ，主串指针 i 不动，模式串指针回退到 `lps[j - 1]`。
4. 如果 `text[i] != pattern[j]` 且 $j == 0$ ，没有可复用的前缀，主串指针 i 右移一位。

整个过程中， i 永远不会左移。KMP 的复杂度是：

- 预处理 LPS: $O(m)$ 。
- 搜索: $O(n)$ 。
- 总时间复杂度: $O(n + m)$ 。
- 额外空间复杂度: $O(m)$ 。

```

def kmp_search(text: str, pattern: str) -> list[int]:
    """ 返回 pattern 在 text 中出现的所有起始下标，支持重叠匹配。 """
    n, m = len(text), len(pattern)

```

```

if m == 0:
    return list(range(n + 1))

lps = build_lps(pattern)
matches = []

i = 0 # text 指针
j = 0 # pattern 指针

while i < n:
    if text[i] == pattern[j]:
        i += 1
        j += 1

        if j == m:
            matches.append(i - j)
            j = lps[j - 1]
    elif j > 0:
        j = lps[j - 1]
    else:
        i += 1

return matches

```

```
kmp_search("ABABDABACDABABCABAB", "ABABCABAB")
```

```

# 重叠匹配示例
kmp_search("AAAAA", "AAA")

```

KMP 搜索实现

6 并查集

并查集 (Disjoint Set Union, DSU), 也常被称为 Union-Find, 是一种用于维护若干个互不相交集合的数据结构。

它主要支持两类操作: - `find(x)`: 查询元素 `x` 所在集合的代表元素; - `union(x, y)`: 合并元素 `x` 与元素 `y` 所在的两个集合。

如果两个元素的代表元素相同, 就说明它们属于同一个集合。因此, 并查集特别适合处理“连通性”问题, 例如判断两个点是否连通、统计连通分量个数、Kruskal 最小生成树、朋友圈/社交关系分组等问题。

6.1 基本思想

并查集通常把每个集合组织成一棵树。树中的每个节点表示一个元素，每个节点保存一个父节点指针 `parent`。

对于一个集合，我们选取树根作为这个集合的代表元素。树根的父节点是它自己，即：

```
parent[root] == root
```

例如，假设有 5 个元素，初始时每个元素单独构成一个集合：

```
{0}, {1}, {2}, {3}, {4}
```

此时每个元素都是自己的代表元素。若执行 `union(0, 1)` 和 `union(1, 2)`，那么 0, 1, 2 会被合并到同一个集合中：

```
{0, 1, 2}, {3}, {4}
```

这时 `find(0)`、`find(1)`、`find(2)` 应该返回同一个代表元素。代表元素具体是谁并不重要，重要的是同一集合中的所有元素经过 `find` 后结果一致。

6.2 两个核心优化

如果直接用树来表示集合，树可能退化成一条链，此时 `find` 的时间复杂度可能达到 $O(N)$ 。为了避免这种情况，实际实现中通常使用两个优化。

6.2.1 1. 路径压缩

在执行 `find(x)` 时，如果一路向上找到了根节点，就顺手把路径上的节点直接挂到根节点下面。

这样以后再查询这些节点时，就可以更快地找到代表元素。

6.2.2 2. 按大小或按秩合并

在执行 `union(x, y)` 时，不随意把一棵树接到另一棵树下面，而是把较小的树接到较大的树下面。这样可以尽量避免树变高。

常见做法有两种：- 按大小合并 (`union by size`)：记录每棵树包含多少个节点；- 按秩合并 (`union by rank`)：记录树高的近似上界。

下面的实现使用按大小合并。

6.3 时间复杂度

同时使用路径压缩和按大小/按秩合并后，并查集的单次操作均摊时间复杂度为：

$$O(\alpha(N))$$

其中 $\alpha(N)$ 是反阿克曼函数。它增长极其缓慢，在实际数据规模下几乎可以看作常数。因此，我们通常认为并查集的 `find` 和 `union` 操作接近 $O(1)$ 。

空间复杂度为 $O(N)$ ，主要用于保存每个元素的父节点和集合大小。

```
class DisjointSetUnion:
    """ 并查集，支持路径压缩和按集合大小合并。 """

    def __init__(self, n: int):
        if n < 0:
            raise ValueError("n must be non-negative")

        self.parent = list(range(n))
        self.size = [1] * n
        self.count = n # 当前连通分量/集合数量

    def __len__(self) -> int:
        return len(self.parent)

    def _validate(self, x: int) -> None:
        if not 0 <= x < len(self.parent):
            raise IndexError(f"index {x} is out of range")

    def find(self, x: int) -> int:
        """ 返回 x 所在集合的代表元素。 """
        self._validate(x)

        if self.parent[x] != x:
            self.parent[x] = self.find(self.parent[x]) # 路径压缩
        return self.parent[x]

    def union(self, x: int, y: int) -> bool:
        """
        合并 x 和 y 所在的集合。

        如果原本不在同一个集合，合并后返回 True;
        如果原本已经在同一个集合，返回 False。
        """
        root_x = self.find(x)
        root_y = self.find(y)

        if root_x == root_y:
            return False

        # 按大小合并：让小集合的根指向大集合的根
        if self.size[root_x] < self.size[root_y]:
            root_x, root_y = root_y, root_x

        self.parent[root_y] = root_x
        self.size[root_x] += self.size[root_y]
        self.count -= 1
```

```

    return True

def connected(self, x: int, y: int) -> bool:
    """ 判断  $x$  和  $y$  是否属于同一个集合。 """
    return self.find(x) == self.find(y)

def component_size(self, x: int) -> int:
    """ 返回  $x$  所在集合的元素个数。 """
    return self.size[self.find(x)]

def components(self) -> dict[int, list[int]]:
    """ 返回当前所有集合，键为代表元素，值为该集合中的元素列表。 """
    groups: dict[int, list[int]] = {}
    for x in range(len(self.parent)):
        root = self.find(x)
        groups.setdefault(root, []).append(x)
    return groups

```

6.4 使用示例

假设有编号为 0 到 6 的 7 个元素。下面通过若干次合并操作来维护它们之间的连通关系。

```

dsu = DisjointSetUnion(7)

dsu.union(0, 1)
dsu.union(1, 2)
dsu.union(3, 4)
dsu.union(5, 6)

print(dsu.connected(0, 2)) # True
print(dsu.connected(0, 4)) # False
print(dsu.component_size(1)) # 3
print(dsu.count) # 3
print(dsu.components())

```

继续执行 `union(2, 4)`，会把 `{0, 1, 2}` 和 `{3, 4}` 合并成一个更大的集合。

```

dsu.union(2, 4)

print(dsu.connected(0, 4)) # True
print(dsu.component_size(3)) # 5
print(dsu.count) # 2
print(dsu.components())

```

6.5 典型应用：统计无向图的连通分量

给定一个无向图，若图中有 n 个顶点和若干条边，可以把每条边 (u, v) 看作一次合并操作 $\text{union}(u, v)$ 。所有边处理完后，并查集中的集合数量就是图的连通分量数量。

```
def count_components(n: int, edges: list[tuple[int, int]]) -> int:
    dsu = DisjointSetUnion(n)
    for u, v in edges:
        dsu.union(u, v)
    return dsu.count

edges = [(0, 1), (1, 2), (3, 4), (5, 6)]
print(count_components(7, edges)) # 3
```

6.6 小结

并查集适合解决动态合并集合、判断元素是否属于同一集合的问题。

它的关键点是：- 用树表示集合，树根作为集合代表；- **find** 用于查找代表元素；- **union** 用于合并两个集合；- 路径压缩可以降低后续查询成本；- 按大小/按秩合并可以避免树退化；- 优化后的并查集在实践中几乎可以看作常数时间。

7 图算法

图是一种应用广泛的数据结构，可以用来表示城市之间的道路、课程之间的先修关系、社交网络中的关注关系等。形式化地说，图由节点集合 V 和边集合 E 组成，记作 (V, E) 。边可以看成节点集合上的二元关系 $(u, v) \in E \subseteq V \times V$ 。

如果边没有方向， (u, v) 和 (v, u) 表示同一条连接，这样的图称为无向图；如果边有方向， (u, v) 表示从 u 指向 v ，这样的图称为有向图。边还可以带权重，用来表示距离、费用、时间等代价。

下面先用邻接表实现一个 **Graph** 类。邻接表把每个节点映射到它的邻居集合，适合表示稀疏图；相比邻接矩阵，它通常能节省空间，并且可以较方便地遍历某个节点的所有出边。

```
class Graph:
    def __init__(self, directed=False):
        """
        初始化图
        :param directed: 是否为有向图，默认是无向图
        """
        self.directed = directed
        self.adj = {} # 邻接表: {节点: {相邻节点: 权重}}

    def add_vertex(self, vertex):
```

```
    """
    添加节点
    """
    if vertex not in self.adj:
        self.adj[vertex] = {}

def remove_vertex(self, vertex):
    """
    删除节点
    """
    if vertex not in self.adj:
        return

    # 删除该节点本身
    del self.adj[vertex]

    # 删除其他节点中指向该节点的边
    for v in self.adj:
        self.adj[v].pop(vertex, None)

def add_edge(self, u, v, weight=1):
    """
    添加边
    :param u: 起点
    :param v: 终点
    :param weight: 边的权重, 默认是 1
    """
    self.add_vertex(u)
    self.add_vertex(v)

    self.adj[u][v] = weight

    # 如果是无向图, 需要反向也加一条边
    if not self.directed:
        self.adj[v][u] = weight

def remove_edge(self, u, v):
    """
    删除边
    """
    if u in self.adj:
        self.adj[u].pop(v, None)

    if not self.directed and v in self.adj:
        self.adj[v].pop(u, None)

def has_vertex(self, vertex):
    """
    判断节点是否存在
```

```
    """
    return vertex in self.adj

def has_edge(self, u, v):
    """
    判断边是否存在
    """
    return u in self.adj and v in self.adj[u]

def get_vertices(self):
    """
    获取所有节点
    """
    return list(self.adj.keys())

def get_neighbors(self, vertex):
    """
    获取某个节点的所有邻居
    """
    if vertex not in self.adj:
        return []

    return list(self.adj[vertex].keys())

def get_edge_weight(self, u, v):
    """
    获取边的权重
    """
    if self.has_edge(u, v):
        return self.adj[u][v]

    return None

def get_edges(self):
    """
    获取所有边
    """
    edges = []

    for u in self.adj:
        for v, weight in self.adj[u].items():
            if self.directed:
                edges.append((u, v, weight))
            else:
                # 避免无向图重复输出边
                if (v, u, weight) not in edges:
                    edges.append((u, v, weight))

    return edges
```

```
def clear(self):
    """
    清空图
    """
    self.adj.clear()

def __str__(self):
    """
    打印图
    """
    return str(self.adj)
```

7.1 BFS 与 DFS

BFS（广度优先搜索）和 DFS（深度优先搜索）都是从一个起始节点出发，不断访问尚未访问过的邻居节点。二者都需要一个 `visited` 集合，避免在有环图中反复访问同一个节点。

它们的核心差异在于待访问节点集合的取出顺序。BFS 使用队列：先发现的节点先被扩展，因此遍历会一层一层向外推进；在无权图中，BFS 常用于求从起点到其他节点的最短边数。DFS 使用栈：后发现的节点先被扩展，因此会沿着一条路径尽量深入，走不下去后再回退；DFS 常用于连通性判断、环检测和生成遍历序列。

下面的 `bfs` 和 `dfs` 函数接收一个 `Graph` 对象和起始节点，并返回实际访问到的节点顺序。

```
from collections import deque

def bfs(graph, start):
    """
    广度优先遍历 BFS
    :param graph: Graph 对象
    :param start: 起始节点
    :return: 遍历顺序列表
    """
    if not graph.has_vertex(start):
        raise ValueError(f"The graph has no node {start}!")

    res = []
    frontiers = deque([start])
    visited = {start}

    while frontiers:
        vertex = frontiers.popleft()
        res.append(vertex)

        for neighbor in graph.get_neighbors(vertex):
```

```

        if neighbor in visited:
            continue

        visited.add(neighbor)
        frontiers.append(neighbor)

    return res

def dfs(graph, start):
    """
    深度优先遍历 DFS
    :param graph: Graph 对象
    :param start: 起始节点
    :return: 遍历顺序列表
    """
    if not graph.has_vertex(start):
        raise ValueError(f"The graph has no node {start}!")

    res = []
    frontiers = [start]
    visited = {start}

    while frontiers:
        vertex = frontiers.pop()
        res.append(vertex)

        for neighbor in reversed(graph.get_neighbors(vertex)):
            if neighbor in visited:
                continue

            visited.add(neighbor)
            frontiers.append(neighbor)

    return res

```

```

graph = Graph()
for u, v in [("A", "B"), ("A", "C"), ("B", "D"), ("C", "D")]:
    graph.add_edge(u, v)

print("BFS:", bfs(graph, "A"))
print("DFS:", dfs(graph, "A"))

```

输出:

```

BFS: ['A', 'B', 'C', 'D']
DFS: ['A', 'B', 'D', 'C']

```


7.2 拓扑排序

拓扑排序是针对有向无环图 (DAG) 的一种排序方式。它把图中的所有顶点排成一个线性序列, 使得对于图中每一条有向边 (u, v) , 顶点 u 一定排在 v 前面。换句话说, 拓扑序描述的是一种满足所有依赖关系的执行顺序, 例如课程先修关系、任务调度、编译依赖等。

如果图中存在有向环, 就不存在合法的拓扑序。原因是环上的每个节点都依赖环中的另一个节点, 无法找到一个真正可以排在最前面的节点。因此, 拓扑排序不仅能给出顺序, 也可以用来判断有向图中是否存在环。

Kahn 算法是一种常用的拓扑排序算法, 核心思想是不断删除当前没有前置依赖的节点。具体流程如下:

1. 先统计每个顶点的入度。入度表示有多少条边指向该顶点, 也就是它还有多少个前置依赖。
2. 把所有入度为 0 的顶点加入队列。它们没有未完成的前置依赖, 可以最先进入拓扑序。
3. 每次从队列中取出一个顶点, 将它加入结果序列; 然后“删除”它发出的所有边, 也就是把它所有后继节点的入度减 1。
4. 如果某个后继节点的入度因此变为 0, 说明它的所有前置依赖都已经处理完, 就把它加入队列。
5. 重复以上过程, 直到队列为空。如果最终结果序列包含了图中的所有顶点, 说明排序成功; 否则说明图中仍有节点无法被删除, 图中存在有向环。

下面的 `topological_sort` 函数接收一个有向 `Graph` 对象, 并返回拓扑序列表。

```
from collections import deque

def topological_sort(graph):
    """
    使用 Kahn 算法进行拓扑排序
    :param graph: 有向 Graph 对象
    :return: 拓扑序列表
    """
    if not graph.directed:
        raise ValueError("Topological sort is only defined for directed graphs.")

    in_degree = {vertex: 0 for vertex in graph.adj}
    for u in graph.adj:
        for v in graph.adj[u]:
            in_degree[v] += 1

    frontiers = deque(vertex for vertex in graph.adj if in_degree[vertex] == 0)
    order = []

    while frontiers:
        vertex = frontiers.popleft()
        order.append(vertex)
```

```

    for neighbor in graph.get_neighbors(vertex):
        in_degree[neighbor] -= 1
        if in_degree[neighbor] == 0:
            frontiers.append(neighbor)

    if len(order) != len(graph.adj):
        raise ValueError("The graph contains a cycle; topological sort is not possible.")

    return order

```

```

dag = Graph(directed=True)
for u, v in [("A", "C"), ("B", "C"), ("B", "D"), ("C", "E"), ("D", "E")]:
    dag.add_edge(u, v)

print("Topological sort:", topological_sort(dag))

```

输出：

```
Topological sort: ['A', 'B', 'C', 'D', 'E']
```

7.3 连通分量与强连通分量

无向图里，如果任意两个顶点都互相可达，它们就在同一个连通分量里；这也常被叫作连通分支。图被分成若干个连通分量后，不同分量之间彼此不连通。

对有向图来说，常见概念有两类：

- 弱连通分量：把边的方向忽略后得到的连通分量。
- 强连通分量（SCC）：在有向图中，若任意两个顶点 u 和 v 都满足 u 可达 v 且 v 可达 u ，它们就在同一个强连通分量里。

无向图的连通分量，以及有向图的弱连通分量，都可以直接用一次 BFS 或 DFS 求出。真正需要专门算法的是强连通分量：它把有向图压缩后会形成一张 DAG，因此常用于依赖分析、环检测、程序分析和网络分群。

求强连通分量的经典算法都能在 $O(V + E)$ 时间内完成：

- Kosaraju 算法：两次 DFS。第一次记录完成时间，第二次在反图上按逆序搜索。实现直观，代码最容易讲清楚。
- Tarjan 算法：一次 DFS，维护 `dfn/lowlink` 和一个栈。它是最常见的单遍 SCC 算法之一。
- Gabow 算法：一次 DFS，维护两个栈 S 和 P 。它不显式使用 `lowlink`，思路和 Tarjan 不同，但同样是线性时间。

下面给出三个算法的函数实现，返回值都是若干个强连通分量，每个分量是一个节点列表。

```
def kosaraju_scc(graph):
    """
    使用 Kosaraju 算法求强连通分量
    :param graph: 有向 Graph 对象
    :return: 若干个强连通分量，每个分量是一个节点列表
    """
    if not graph.directed:
        raise ValueError("Strongly connected components are only defined for directed graphs.")

    visited = set()
    order = []

    def dfs1(vertex):
        visited.add(vertex)
        for neighbor in graph.get_neighbors(vertex):
            if neighbor not in visited:
                dfs1(neighbor)
        order.append(vertex)

    for vertex in graph.get_vertices():
        if vertex not in visited:
            dfs1(vertex)

    reverse_adj = {vertex: [] for vertex in graph.get_vertices()}
    for u in graph.adj:
        for v in graph.adj[u]:
            reverse_adj[v].append(u)

    visited.clear()
    sccs = []

    def dfs2(vertex, component):
        visited.add(vertex)
        component.append(vertex)
        for neighbor in reverse_adj.get(vertex, []):
            if neighbor not in visited:
                dfs2(neighbor, component)

    for vertex in reversed(order):
        if vertex not in visited:
            component = []
            dfs2(vertex, component)
            sccs.append(component)

    return sccs

def tarjan_scc(graph):
```

```

"""
使用 Tarjan 算法求强连通分量
:param graph: 有向 Graph 对象
:return: 若干个强连通分量, 每个分量是一个节点列表
"""
if not graph.directed:
    raise ValueError("Strongly connected components are only defined for
        directed graphs.")

index = 0
indices = {}
lowlink = {}
stack = []
on_stack = set()
sccs = []

def dfs(vertex):
    nonlocal index
    indices[vertex] = index
    lowlink[vertex] = index
    index += 1
    stack.append(vertex)
    on_stack.add(vertex)

    for neighbor in graph.get_neighbors(vertex):
        if neighbor not in indices:
            dfs(neighbor)
        lowlink[vertex] = min(lowlink[vertex], lowlink[neighbor])
        elif neighbor in on_stack:
            lowlink[vertex] = min(lowlink[vertex], indices[neighbor])

    if lowlink[vertex] == indices[vertex]:
        component = []
        while True:
            node = stack.pop()
            on_stack.remove(node)
            component.append(node)
            if node == vertex:
                break
        sccs.append(component)

for vertex in graph.get_vertices():
    if vertex not in indices:
        dfs(vertex)

return sccs

def gabow_scc(graph):

```

```

"""
使用 Gabow 算法求强连通分量
:param graph: 有向 Graph 对象
:return: 若干个强连通分量, 每个分量是一个节点列表
"""
if not graph.directed:
    raise ValueError("Strongly connected components are only defined for
        directed graphs.")

preorder = {}
assigned = set()
stack_s = []
stack_p = []
order = 0
sccs = []

def dfs(vertex):
    nonlocal order
    preorder[vertex] = order
    order += 1
    stack_s.append(vertex)
    stack_p.append(vertex)

    for neighbor in graph.get_neighbors(vertex):
        if neighbor not in preorder:
            dfs(neighbor)
        elif neighbor not in assigned:
            while preorder[stack_p[-1]] > preorder[neighbor]:
                stack_p.pop()

    if stack_p and stack_p[-1] == vertex:
        stack_p.pop()
        component = []
        while True:
            node = stack_s.pop()
            assigned.add(node)
            component.append(node)
            if node == vertex:
                break
        sccs.append(component)

for vertex in graph.get_vertices():
    if vertex not in preorder:
        dfs(vertex)

return sccs

```

7.4 最短路径

图的最短路径问题关心的是：从一个起点到一个终点，如何找到总代价最小的路径。这里的代价通常是边权之和，也可以表示距离、时间、费用等。

最短路径问题常分为两类：

- 单源最短路径：给定一个起点，求它到所有顶点的最短距离。
- 多源最短路径：求任意两点之间的最短距离。

7.4.1 Dijkstra 算法

Dijkstra 算法是经典的单源最短路径算法，适用于**边权非负**的图。它的核心思想是贪心：每次从尚未确定最短路的顶点中，取出当前距离最小的顶点，并用它去松弛相邻顶点。由于边权非负，一旦某个顶点被取出，它的最短距离就不会再被更新。

适用条件：

- 可以处理有向图或无向图。
- 边权必须非负。

复杂度分析：

- 使用二叉堆时，时间复杂度为 $O((V + E) \log V)$ 。
- 空间复杂度为 $O(V + E)$ 。

7.4.2 Bellman-Ford 算法

Bellman-Ford 算法也是单源最短路径算法，但它允许存在**负权边**。它的核心思想是反复对所有边做松弛操作，总共重复 $V - 1$ 轮。因为任意最短路径最多只会经过 $V - 1$ 条边，所以做完这 $V - 1$ 轮后，最短距离通常就已经稳定。

它还可以用于检测**负权环**：如果再额外做一轮松弛，距离仍然还能变小，就说明图中存在从起点可达的负权环，最短路不再有定义。

适用条件：

- 可以处理有向图或无向图。
- 可以包含负权边。
- 不能存在从源点可达的负权环。

复杂度分析：

- 时间复杂度为 $O(VE)$ 。
- 空间复杂度为 $O(V)$ ，如果只保留距离和前驱数组的话。

7.4.3 Floyd 算法

Floyd 算法是典型的多源最短路径算法，也就是一次性求出任意两点之间的最短距离。它使用动态规划思想，依次枚举中转点 k ，判断路径 $i \rightarrow k \rightarrow j$ 是否能比当前的 $i \rightarrow j$ 更短。

适用条件：

- 可以处理有向图或无向图。
- 可以存在负权边，但不能存在负权环。

复杂度分析：

- 时间复杂度为 $O(V^3)$ 。
- 空间复杂度为 $O(V^2)$ 。

一般来说，Dijkstra 更适合单源查询和稀疏图，Floyd 更适合顶点数不大但需要频繁查询任意两点最短路的场景。

```
from heapq import heappop, heappush

def dijkstra(graph, start):
    """
    使用 Dijkstra 算法求单源最短路径
    :param graph: Graph 对象
    :param start: 起始节点
    :return: dist, prev
    """
    if not graph.has_vertex(start):
        raise ValueError(f"The graph has no node {start}!")

    dist = {v: float('inf') for v in graph.get_vertices()}
    prev = {v: None for v in graph.get_vertices()}
    dist[start] = 0
    pq = [(0, start)]

    while pq:
        cur_dist, u = heappop(pq)
        if cur_dist != dist[u]:
            continue

        for v in graph.get_neighbors(u):
            weight = graph.get_edge_weight(u, v)
            if weight is None:
                continue
            if weight < 0:
                raise ValueError("Dijkstra algorithm requires non-negative edge weights.")

            nd = cur_dist + weight
            if nd < dist[v]:
                dist[v] = nd
                prev[v] = u
                heappush(pq, (nd, v))

    return dist, prev
```

```

def bellman_ford(graph, start):
    """
    使用 Bellman-Ford 算法求单源最短路径
    :param graph: Graph 对象
    :param start: 起始节点
    :return: dist, prev
    """
    if not graph.has_vertex(start):
        raise ValueError(f"The graph has no node {start}!")

    vertices = graph.get_vertices()
    dist = {v: float('inf') for v in vertices}
    prev = {v: None for v in vertices}
    dist[start] = 0

    edges = graph.get_edges()
    if not graph.directed:
        expanded = []
        for u, v, w in edges:
            expanded.append((u, v, w))
            expanded.append((v, u, w))
        edges = expanded

    for _ in range(len(vertices) - 1):
        updated = False
        for u, v, weight in edges:
            if dist[u] == float('inf'):
                continue
            nd = dist[u] + weight
            if nd < dist[v]:
                dist[v] = nd
                prev[v] = u
                updated = True
        if not updated:
            break

    for u, v, weight in edges:
        if dist[u] != float('inf') and dist[u] + weight < dist[v]:
            raise ValueError("The graph contains a negative cycle reachable from the source.")

    return dist, prev


def floyd(graph):
    """
    使用 Floyd 算法求任意两点最短路径
    :param graph: Graph 对象

```



```

: return: dist, nxt
"""
vertices = graph.get_vertices()
dist = {u: {v: float('inf') for v in vertices} for u in vertices}
nxt = {u: {v: None for v in vertices} for u in vertices}

for v in vertices:
    dist[v][v] = 0
    nxt[v][v] = v

for u in graph.adj:
    for v, weight in graph.adj[u].items():
        if weight < dist[u][v]:
            dist[u][v] = weight
            nxt[u][v] = v

for k in vertices:
    for i in vertices:
        if dist[i][k] == float('inf'):
            continue
        for j in vertices:
            nd = dist[i][k] + dist[k][j]
            if nd < dist[i][j]:
                dist[i][j] = nd
                nxt[i][j] = nxt[i][k]

for v in vertices:
    if dist[v][v] < 0:
        raise ValueError("The graph contains a negative cycle; Floyd shortest
            paths are undefined.")

return dist, nxt

def reconstruct_path(nxt, start, end):
    """
    根据 Floyd 的 next 矩阵重建路径
    """
    if nxt[start][end] is None:
        return []

    path = [start]
    while start != end:
        start = nxt[start][end]
        if start is None:
            return []
        path.append(start)

    return path

```

7.5 最小生成树

最小生成树 (Minimum Spanning Tree, MST) 是无向连通带权图中的一个子图。它需要连接图中的所有顶点，并且边权总和最小。因为它是一棵树，所以如果图中有 V 个顶点，那么最小生成树一定包含 $V - 1$ 条边。

最小生成树问题只针对**无向图**讨论。如果原图不连通，就不存在覆盖所有顶点的一棵生成树，只能得到每个连通分量内部的最小生成森林。

最小生成树有几个常用性质：

- 无环性：最小生成树中不会出现环，否则可以删掉环上一条边，仍然保持连通。
- 连通性：最小生成树必须连接所有顶点。
- 边数性质：包含 V 个顶点的生成树一定有 $V - 1$ 条边。
- 切分性质：把顶点集合切成两部分，跨越这个切分的最小权重边一定可以属于某棵最小生成树。
- 环性质：任意一个环中，权重最大的边一定不必出现在某棵最小生成树中。

7.5.1 Prim 算法

Prim 算法从一个起点开始，维护一个已经加入生成树的顶点集合。每一步选择一条连接“树内顶点”和“树外顶点”的最小权重边，把新的顶点加入生成树。它的思路更像是从一个局部连通块不断向外扩张。

复杂度分析：

- 使用二叉堆和邻接表时，时间复杂度为 $O(E \log E)$ ，也常写作 $O(E \log V)$ 。
- 空间复杂度为 $O(V + E)$ 。

7.5.2 Kruskal 算法

Kruskal 算法先把所有边按权重从小到大排序，然后依次尝试加入这些边。只要加入某条边不会形成环，就把它放进生成树。为了高效判断是否成环，通常使用并查集维护顶点所属的连通分量。

复杂度分析：

- 排序所有边需要 $O(E \log E)$ 时间，并查集操作几乎是常数时间，因此总时间复杂度为 $O(E \log E)$ 。
- 空间复杂度为 $O(V + E)$ 。

Prim 更适合从某个顶点开始逐步扩展；Kruskal 更适合从全局边集出发，按边权从小到大构造生成树。

```
from heapq import heappop, heappush
```

```
def prim_mst(graph, start=None):
```

```
    """
```

```
    使用 Prim 算法求最小生成树
```

```
    :param graph: 无向 Graph 对象
```

```

:param start: 起始节点, 默认使用第一个节点
:return: mst_edges, total_weight
"""
if graph.directed:
    raise ValueError("Minimum spanning tree is only defined for undirected
graphs.")

vertices = graph.get_vertices()
if not vertices:
    return [], 0

if start is None:
    start = vertices[0]
if not graph.has_vertex(start):
    raise ValueError(f"The graph has no node {start}!")

visited = {start}
pq = []
mst_edges = []
total_weight = 0

for neighbor in graph.get_neighbors(start):
    weight = graph.get_edge_weight(start, neighbor)
    heappush(pq, (weight, start, neighbor))

while pq and len(visited) < len(vertices):
    weight, u, v = heappop(pq)
    if v in visited:
        continue

    visited.add(v)
    mst_edges.append((u, v, weight))
    total_weight += weight

    for neighbor in graph.get_neighbors(v):
        if neighbor not in visited:
            edge_weight = graph.get_edge_weight(v, neighbor)
            heappush(pq, (edge_weight, v, neighbor))

if len(visited) != len(vertices):
    raise ValueError("The graph is not connected; a spanning tree does not
exist.")

return mst_edges, total_weight

def kruskal_mst(graph):
    """
    使用 Kruskal 算法求最小生成树

```

```

:param graph: 无向 Graph 对象
:return: mst_edges, total_weight
"""
if graph.directed:
    raise ValueError("Minimum spanning tree is only defined for undirected
graphs.")

parent = {}
rank = {}

def make_set(x):
    parent[x] = x
    rank[x] = 0

def find(x):
    if parent[x] != x:
        parent[x] = find(parent[x])
    return parent[x]

def union(x, y):
    root_x = find(x)
    root_y = find(y)
    if root_x == root_y:
        return False

    if rank[root_x] < rank[root_y]:
        parent[root_x] = root_y
    elif rank[root_x] > rank[root_y]:
        parent[root_y] = root_x
    else:
        parent[root_y] = root_x
        rank[root_x] += 1

    return True

vertices = graph.get_vertices()
for vertex in vertices:
    make_set(vertex)

mst_edges = []
total_weight = 0
edges = sorted(graph.get_edges(), key=lambda edge: edge[2])

for u, v, weight in edges:
    if union(u, v):
        mst_edges.append((u, v, weight))
        total_weight += weight
    if len(mst_edges) == len(vertices) - 1:
        break

```

```
if len(mst_edges) != len(vertices) - 1:
    raise ValueError("The graph is not connected; a spanning tree does not exist.")

return mst_edges, total_weight
```